

Universidad Tecnológica de Pereira

Facultad de Ingenierías

Programa de Ingeniería de Sistemas y Computación

HPC

Implementación y Análisis de un Autómata Celular para el Tráfico Vehicular

Presentado por:

Juan Esteban Cardona Blandón

Jhoan Esteban Corrales Rodríguez

Sebastian Perdomo

Juana Valentina Martínez

Profesor:

Ramiro Andrés Barrios Valencia

Pereira, Colombia

14 de mayo de 2026

Índice

1. Marco Teórico	3
1.1. Computación de Alto Rendimiento (HPC)	3
1.2. Programación Secuencial y Concurrente	3
1.3. Sistemas de Memoria Compartida	3
1.4. Jerarquía de Caché, Líneas de Caché y Localidad de Memoria	4
1.5. Métricas de Rendimiento en Computación Secuencial	4
1.6. Métricas de Rendimiento en Computación Paralela	5
1.7. OpenMP como Modelo de Programación Paralela	6
1.8. Autómatas Celulares	7
1.9. Modelo de Nagel-Schreckenberg	8
1.10. Transición de Fase Libre-Congestionado	9
1.11. Intensidad Aritmética y Modelo Roofline	9
2. Marco Conceptual	10
2.1. Características del PC	10
3. Desarrollo de la implementación	11
3.1. Modelo del autómata celular de tráfico	11
3.2. Velocidad media y estado estacionario	12
3.3. Criterio de convergencia del calentamiento	12
3.4. Arquitectura de datos	13
3.5. Implementación secuencial	14
3.6. Implementación paralela con OpenMP	14
3.7. Optimización por compilador	15
3.8. Optimización de memoria	16
3.8.1. MEM-1: Reducción del tipo de celda a <code>uint8_t</code>	16
3.8.2. MEM-2: Fusión de pasadas y eliminación del módulo	17
3.8.3. MEM-3: Alineación de buffers a línea de caché	17
3.9. Matriz de variantes de compilación	17
3.10. Diseño del script de pruebas	18
3.11. Matriz de configuraciones	18
3.12. Techo de calentamiento adaptativo	18
3.13. Formato del CSV de resultados	19

4. Perfilado de código	19
4.1. Binarios compilados	19
4.2. gprof: perfilado por instrumentación del compilador	20
4.3. perf stat: contadores de hardware de la PMU	20
4.4. perf record: muestreo basado en interrupciones de hardware	21
4.5. Valgrind Massif: perfilado de uso del heap	22
4.6. Valgrind Cachegrind: simulación de la jerarquía de caché	22
4.7. Visión de conjunto	23
4.8. Resultados del perfilado	23
4.8.1. Localización del hotspot	24
4.8.2. Estabilidad temporal y throughput	26
4.8.3. Limitaciones de hardware	27
4.8.4. Accesos a memoria por nivel	29
5. Pruebas	29
5.1. Resultados de los algoritmos secuenciales	30
5.2. Resultados de la implementación con OpenMP	34
5.3. Resultados de la implementación con OpenMP con optimización por compilador	37
5.4. Resultados de la implementación con OpenMP con optimización de memoria y compilador	41
6. Conclusiones	44

1. Marco Teórico

1.1. Computación de Alto Rendimiento (HPC)

La Computación de Alto Rendimiento (HPC, por sus siglas en inglés) consiste en la agregación de potencia de cálculo a través de arquitecturas paralelas para resolver problemas científicos y tecnológicos de extrema complejidad. Esta disciplina permite ejecutar algoritmos y modelados matemáticos que resultarían inabarcables para un sistema tradicional debido a restricciones de tiempo y recursos de hardware (Hager & Wellein, 2010). Al dividir cargas masivas de trabajo en instrucciones más pequeñas que se procesan simultáneamente, HPC maximiza la velocidad de procesamiento de datos y la obtención de resultados en la ingeniería (Patterson & Hennessy, 2017).

1.2. Programación Secuencial y Concurrente

La programación secuencial es el paradigma clásico de desarrollo donde las instrucciones de un algoritmo se ejecutan de forma estrictamente lineal, una tras otra, sobre un único núcleo de procesamiento. En este modelo tradicional, el límite de rendimiento está directamente dictado por la frecuencia de reloj del hardware. Como respuesta a estas limitaciones físicas, la programación concurrente estructura el software en múltiples hilos o tareas independientes que hacen progreso de forma superpuesta en el tiempo. Cuando estas tareas operan de manera simultánea en una arquitectura de hardware multinúcleo, la concurrencia se traduce en paralelismo físico, lo cual permite dividir la carga de trabajo y reducir drásticamente el tiempo de ejecución en problemas de cómputo intensivo (Ben-Ari, 2006).

1.3. Sistemas de Memoria Compartida

En el ámbito de la computación paralela, el modelo de memoria compartida establece que múltiples hilos de ejecución (como los creados por la librería Pthreads) operan dentro de un mismo espacio de direccionamiento lógico. A nivel de arquitectura, esto significa que todos los núcleos del procesador pueden acceder de forma directa y simultánea a las mismas estructuras de datos residentes en la memoria principal. Para problemas matriciales, este modelo resulta excepcionalmente eficiente frente a los sistemas de memoria distribuida, ya que evita la necesidad de copiar y transmitir los datos de las matrices de origen repetidamente entre procesos. Aunque este paradigma exige control riguroso para evitar condiciones de carrera, en operaciones intrínsecamente independientes como el cálculo individual de las celdas de una matriz resultado, su implementación maximiza la eficiencia computacional sin sobrecarga de sincronización (Silberschatz et al., 2018).

1.4. Jerarquía de Caché, Líneas de Caché y Localidad de Memoria

El rendimiento de una aplicación de alto rendimiento (HPC) no depende exclusivamente de la capacidad matemática del procesador, sino también del acceso al subsistema de memoria. Para mitigar la alta latencia térmica y física de la memoria RAM, los procesadores modernos integran una jerarquía de memoria ultrarrápida cercana a los núcleos, conocida como memoria Caché, habitualmente dividida en niveles (L1, L2 y L3). La unidad mínima de transferencia de información entre la RAM principal y la memoria caché no es un byte aislado, sino un bloque continuo de datos denominado línea de caché (*cache line*), que en las arquitecturas modernas suele constar de 64 bytes (Drepper, 2007).

Esta transferencia por bloques está fundamentada en el principio de localidad espacial de memoria, un concepto empírico que dicta que si un programa accede a una dirección de memoria, es inminentemente probable que acceda a las direcciones adyacentes a continuación. En lenguajes de programación como C, las matrices bidimensionales se almacenan en la memoria RAM de manera contigua organizadas por filas (*row-major order*). Durante la multiplicación matricial clásica iterativa, la segunda matriz se recorre inevitablemente por columnas; este patrón de acceso ortogonal rompe por completo la localidad espacial. Al intentar leer datos saltando grandes bloques de memoria, los valores necesarios no se encuentran en la línea de caché cargada, provocando fallos de caché (*cache misses*) que obligan al procesador a detenerse (*stall*) mientras busca los datos nuevamente en la lenta memoria RAM, lo cual penaliza severamente el desempeño del algoritmo (Patterson & Hennessy, 2017).

1.5. Métricas de Rendimiento en Computación Secuencial

El análisis de un algoritmo secuencial requiere establecer una línea base absoluta sobre la cual evaluar cualquier mejora algorítmica o de hardware. La métrica fundamental empírica es el Tiempo de Ejecución (*Wall-clock time*), que representa el tiempo físico real transcurrido desde el inicio hasta la finalización del programa. Sin embargo, dado que este valor fluctúa dependiendo de la interferencia del sistema operativo y los accesos a memoria, la arquitectura de computadores recurre a métricas de rendimiento intrínseco (*throughput*) (Patterson & Hennessy, 2017).

Para cargas de trabajo de alto rendimiento algebraico como la multiplicación de matrices, la métrica estandarizada predominante es el volumen de Operaciones de Punto Flotante por Segundo (FLOPS, por sus siglas en inglés). Esta medida cuantifica el rendimiento matemático bruto del procesador y se define mediante la siguiente relación:

$$\text{FLOPS} = \frac{\text{Operaciones Aritméticas Totales}}{T_{\text{ejecución}}} \quad (1)$$

Para el algoritmo clásico iterativo que multiplica dos matrices cuadradas de dimensión $n \times n$, el número total de operaciones de punto flotante se establece analíticamente en $2n^3$ (contabilizando una instrucción de multiplicación y una de suma por cada iteración del bucle más interno). Al contrastar los FLOPS experimentales obtenidos en la ejecución secuencial contra el rendimiento teórico máximo (*Peak FLOPS*) que el fabricante del procesador garantiza para un solo núcleo, es posible diagnosticar ineficiencias de memoria o cuellos de botella antes de recurrir a la paralelización (Hager & Wellein, 2010).

Adicionalmente, el rendimiento de la ejecución en un único núcleo se evalúa a nivel microarquitectónico mediante el IPC (Instrucciones por Ciclo de reloj). Esta métrica indica la eficiencia con la que el código compilado logra mantener ocupada la segmentación (*pipeline*) del procesador, minimizando las detenciones o burbujas producidas por dependencias de datos en la ejecución lineal (Yi et al., 2020).

1.6. Métricas de Rendimiento en Computación Paralela

Para cuantificar el éxito de una implementación paralela y su eficiencia frente a la ejecución secuencial, la literatura científica emplea métricas matemáticas estandarizadas. La métrica fundamental es el *Speedup* o Aceleración (S_p), la cual mide la ganancia de velocidad relativa. Se define formalmente como:

$$S_p = \frac{T_1}{T_p} \quad (2)$$

donde T_1 representa el tiempo de ejecución del algoritmo secuencial óptimo sobre un único núcleo, y T_p es el tiempo de ejecución del algoritmo paralelo empleando p unidades de procesamiento (hilos o procesos). Derivada de esta métrica se encuentra la Eficiencia (E_p), que evalúa el grado de aprovechamiento de los recursos físicos, indicando qué fracción del tiempo los procesadores realizan trabajo útil sin quedar ociosos:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p} \quad (3)$$

El desempeño asintótico de estos sistemas está regido por dos leyes analíticas que abordan la escalabilidad desde diferentes perspectivas teóricas. La **Ley de Amdahl** evalúa la *escalabilidad fuerte*, demostrando que el *Speedup* máximo de un sistema está estrictamente limitado por la fracción de código que no se puede paralelizar. Asumiendo un tamaño de problema fijo, la ley se expresa como:

$$S_{\text{Amdahl}} \leq \frac{1}{(1 - f) + \frac{f}{p}} \quad (4)$$

donde f es la fracción del programa que es paralelizable y $(1 - f)$ es la porción estrictamente secuencial (como la inicialización de memoria mediante `malloc` o la lectura/escritura en disco). Según este postulado matemático, incluso si el número de procesadores p tiende a infinito, la aceleración máxima nunca superará $1/(1 - f)$. Esto explica el rendimiento decreciente observado cuando el uso de múltiples hilos lógicos en un procesador alcanza su límite físico y no aporta mejoras significativas (Patterson & Hennessy, 2017).

En contraste, la **Ley de Gustafson-Barsis** describe la *escalabilidad débil*. Esta ley argumenta que, en la práctica profesional, los ingenieros no buscan resolver un problema fijo más rápido, sino que utilizan el mayor poder de cómputo para resolver problemas de un tamaño sustancialmente mayor en el mismo marco de tiempo (Gustafson, 1988). Su modelo se define como:

$$S_{\text{Gustafson}} = (1 - f) + p \cdot f \quad (5)$$

Bajo la visión de Gustafson, a medida que la dimensión de las matrices crece de forma masiva, la cantidad de operaciones paralelas f (multiplicaciones flotantes) aumenta drásticamente, haciendo que la fracción secuencial inicial $(1 - f)$ pierda peso relativo. Esto mitiga el cuello de botella pronosticado por Amdahl, permitiendo alcanzar un *Speedup* escalable y un uso altamente eficiente de los recursos al ajustar la carga de trabajo proporcionalmente al hardware.

1.7. OpenMP como Modelo de Programación Paralela

OpenMP (*Open Multi-Processing*) es una interfaz de programación de aplicaciones (API) de estándar abierto para la programación paralela en sistemas de memoria compartida, desarrollada y mantenida por el consorcio OpenMP Architecture Review Board. El estándar define un conjunto de directivas de compilador, rutinas de biblioteca en tiempo de ejecución y variables de entorno que permiten al programador expresar paralelismo a nivel de bucles y tareas directamente sobre un código fuente C, C++ o Fortran existente, sin necesidad de reescribir la arquitectura del programa (Dagum & Menon, 1998). Esta característica lo convierte en el modelo predominante para la paralelización incremental en HPC sobre arquitecturas multinúcleo de memoria compartida.

El modelo de ejecución de OpenMP se fundamenta en el paradigma de bifurcación y unión (*fork-join*). El programa inicia su ejecución como un hilo único denominado hilo maestro, el cual ejecuta el código secuencial de forma ordinaria. Al encontrar una directiva de región paralela, el hilo maestro bifurca un equipo de hilos de trabajo que ejecutan el bloque de código encerrado de forma concurrente; al concluir dicha región, todos los hilos se sincronizan en

una barrera implícita y se unen de nuevo al hilo maestro, que retoma la ejecución secuencial (Dagum & Menon, 1998). Este ciclo puede repetirse tantas veces como regiones paralelas declare el programa, lo que permite una adopción gradual y controlada del paralelismo sobre una base de código secuencial ya validada.

La directiva central para la paralelización de bucles es `#pragma omp parallel for`, que distribuye automáticamente las iteraciones del bucle entre los hilos del equipo. El programador puede controlar la política de distribución de trabajo mediante la cláusula `schedule`, que admite los modos estático (*static*), dinámico (*dynamic*) y guiado (*guided*), cada uno con implicaciones distintas sobre el balance de carga y la sobrecarga de coordinación entre hilos (Chapman et al., 2008). Para la multiplicación de matrices, el modo estático resulta óptimo dado que las iteraciones presentan una carga de trabajo homogénea; esto permite al compilador asignar bloques contiguos de filas a cada hilo sin incurrir en la sobrecarga de planificación dinámica en tiempo de ejecución.

El modelo de datos de OpenMP clasifica las variables del programa en dos categorías: compartidas (*shared*), accesibles y modificables por todos los hilos del equipo simultáneamente, y privadas (*private*), de las cuales cada hilo posee una copia independiente. El control explícito sobre este ámbito de visibilidad es un requisito de corrección del programa paralelo: las matrices de entrada y la matriz resultado deben declararse como compartidas para que todos los hilos lean y escriban sobre la misma estructura de datos, mientras que las variables de índice de los bucles internos deben ser privadas para evitar condiciones de carrera (Chapman et al., 2008). Dado que cada celda C_{ij} de la matriz resultado es calculada de forma completamente independiente por un único hilo, la multiplicación de matrices no requiere mecanismos de sincronización explícita como secciones críticas o operaciones atómicas, lo que elimina la principal fuente de sobrecarga en implementaciones OpenMP y permite un escalado cercano al teórico.

1.8. Autómatas Celulares

Un autómata celular (AC) es un sistema dinámico discreto distribuido espacialmente en el que tanto el tiempo como el espacio se representan de forma discreta (Wolfram, 1984). Formalmente, un AC unidimensional se define como la tupla $\mathcal{A} = (\mathbb{Z}, Q, \mathcal{N}, f)$, donde $\mathbb{Z}$ es el espacio de celdas indexadas por los enteros, Q es un conjunto finito de estados posibles, $\mathcal{N} = \{-r, \dots, 0, \dots, r\}$ es la vecindad de radio r , y $f : Q^{2r+1} \rightarrow Q$ es la función de transición local. En cada paso de tiempo, esta función se aplica de manera sincrónica e idéntica a todas las celdas:

$$R^{t+1}(i) = f\left(R^t(i-r), \dots, R^t(i), \dots, R^t(i+r)\right).$$

La propiedad que habilita directamente el cómputo paralelo es esta actualización sincrónica: el estado nuevo de la celda i depende exclusivamente del estado de su vecindad en el instante t , sin que intervengan cambios ocurridos durante el mismo paso. En consecuencia, todas las celdas son computacionalmente independientes entre sí dentro de un mismo paso de tiempo, lo que hace del modelo de AC una estructura naturalmente paralelizable sin condiciones de carrera entre hilos.

Wolfram clasificó los AC elementales unidimensionales, aquellos con $|Q| = 2$ y $r = 1$, que generan 256 reglas distintas, en cuatro clases de comportamiento asintótico (Wolfram, 1984). La clase I agrupa autómatas que convergen a un estado homogéneo uniforme independientemente de las condiciones iniciales, análogo a un punto límite en sistemas continuos. La clase II produce estructuras periódicas o estacionarias, análoga a ciclos límite. La clase III exhibe comportamiento caótico semejante al de atractores extraños. La clase IV genera patrones complejos localizados con interacciones no triviales, y se ha conjeturado que es capaz de computación universal, por lo que propiedades de su comportamiento a tiempo infinito son indecidibles.

1.9. Modelo de Nagel-Schreckenberg

El modelo de Nagel y Schreckenberg (NaSch), publicado en 1992, fue el primer autómata celular diseñado explícitamente para simular el flujo vehicular en una vía de un solo carril (Nagel & Schreckenberg, 1992). El modelo opera sobre un arreglo unidimensional de L celdas con condiciones de frontera periódicas, donde cada celda puede estar vacía u ocupada por un vehículo con velocidad entera $v \in \{0, 1, \dots, v_{\text{máx}}\}$. La dinámica de cada paso de tiempo $t \rightarrow t + 1$ se define mediante cuatro reglas aplicadas en paralelo a todos los vehículos simultáneamente:

1. Aceleración: $v_n \leftarrow \min(v_n + 1, v_{\text{máx}})$.
2. Frenado por interacción: $v_n \leftarrow \min(v_n, d_n - 1)$, donde d_n es el número de celdas vacías frente al vehículo n .
3. Aleatorización: con probabilidad p , si $v_n > 0$, entonces $v_n \leftarrow v_n - 1$.
4. Movimiento: el vehículo n avanza v_n celdas.

El parámetro $p \in [0, 1]$ modela la conducta imperfecta del conductor humano y fue la primera formalización del origen estocástico de los atascos espontáneos sin causa aparente en la vía.

La implementación desarrollada en este trabajo corresponde al caso límite determinista con $v_{\text{máx}} = 1$ y $p = 0$. Bajo estas condiciones, la regla de aceleración es trivial dado que todo vehículo alcanza $v = 1$ en el mismo paso, y la aleatorización desaparece, reduciéndose la dinámica a una sola condición binaria: un vehículo avanza si y solo si la celda inmediatamente frente a él estaba vacía en el instante t . Este caso particular es algebraicamente equivalente a la Regla 184 de los AC elementales y constituye el caso base analíticamente tratable del modelo NaSch, estudiado explícitamente por los autores originales (Nagel & Schreckenberg, 1992). La simplificación es justificada porque preserva el comportamiento colectivo esencial del modelo, la transición entre flujo libre y congestionado, mientras elimina los grados de libertad que no son relevantes para el análisis de rendimiento paralelo que motiva este trabajo.

1.10. Transición de Fase Libre-Congestionado

El experimento de barrido de densidades tiene por objetivo observar el cambio cualitativo de comportamiento del sistema en función de la densidad vehicular $\rho = N_{\text{cars}}/N$. Para densidades bajas, la mayoría de los vehículos encuentra espacio libre y la velocidad media tiende a $\bar{v} \approx 1$, correspondiente al régimen de flujo libre. Para densidades altas, los vehículos se bloquean mutuamente y $\bar{v} \rightarrow 0$, lo que define el régimen congestionado. Esta transición entre fases de comportamiento cualitativamente distinto fue uno de los resultados centrales reportados por Nagel y Schreckenberg (Nagel & Schreckenberg, 1992), quienes la denominaron transición de flujo laminar a ondas de arranque-parada.

En el caso determinista con $v_{\text{máx}} = 1$ y $p = 0$, la transición es abrupta y ocurre exactamente en $\rho = 0,5$. Para $\rho \leq 0,5$, todos los vehículos pueden moverse en cada paso y $\bar{v} = 1$; para $\rho > 0,5$, el flujo queda limitado por la densidad de espacios vacíos y $\bar{v} = (1 - \rho)/\rho$. La representación canónica de este fenómeno es el diagrama fundamental, que grafica el flujo vehicular $q = \rho \cdot \bar{v}$ en función de ρ . Para el modelo implementado, el diagrama fundamental adopta una forma triangular simétrica, conocida en la literatura como función tent, con un máximo de $q_{\text{máx}} = 0,5$ en $\rho = 0,5$. Este diagrama es la referencia estándar para validar modelos de tráfico en la literatura, y justifica tanto el rango de densidades elegido en el experimento como el registro de la velocidad media $\bar{v}$ como variable de salida principal del simulador.

1.11. Intensidad Aritmética y Modelo Roofline

El modelo Roofline, propuesto por Williams, Waterman y Patterson (Williams et al., 2009), es un modelo de rendimiento visual que relaciona el desempeño alcanzable de un kernel de cómputo con dos parámetros del hardware: el rendimiento de cómputo pico π (en

operaciones por segundo) y el ancho de banda de memoria pico β (en bytes por segundo). La variable que caracteriza al kernel es la intensidad aritmética:

$$I = \frac{\text{operaciones ejecutadas}}{\text{bytes transferidos desde/hacia memoria principal}}.$$

El desempeño alcanzable queda acotado superiormente por:

$$P \leq \min(\pi, \beta \cdot I).$$

Un kernel es memory-bound si $I < \pi/\beta$, en cuyo caso el cuello de botella es el ancho de banda de memoria y no la capacidad de cómputo; es compute-bound si $I > \pi/\beta$. El punto de cresta (ridge point) en la coordenada $I = \pi/\beta$ marca la frontera entre ambos regímenes.

Para el autómata celular de tráfico implementado, cada celda requiere tres lecturas ($R^t(i-1)$, $R^t(i)$, $R^t(i+1)$) y una escritura ($R^{t+1}(i)$), junto con aproximadamente dos operaciones lógicas para evaluar la regla de transición. Representando cada celda como un entero de un byte, la transferencia de memoria por celda es de 4 bytes y el número de operaciones es de alrededor de 2, lo que produce una intensidad aritmética $I \approx 0,4$ op/byte. Este valor se sitúa por debajo del punto de cresta de cualquier arquitectura multicore contemporánea, clasificando el kernel del autómata como claramente memory-bandwidth bound (Williams et al., 2009).

Esta caracterización tiene una implicación directa para la escalabilidad paralela de la implementación: al incrementar el número de hilos, la demanda agregada de ancho de banda de la memoria compartida se satura antes de que la capacidad de cómputo adicional pueda aprovecharse. En consecuencia, el speedup esperado con p hilos alcanzará una saturación anticipada determinada no por el overhead de sincronización ni por la fracción serial del programa (Ley de Amdahl), sino por el límite físico del ancho de banda de memoria de la arquitectura subyacente.

2. Marco Conceptual

2.1. Características del PC

- Procesador: Ryzen 5 7600X
- Cantidad de núcleos: 6
- Cantidad de subprocesos/hilos: 12

- Frecuencia básica del procesador: 4.7GHz
- Frecuencia Turbo máxima: 5.3GHz
- Memoria RAM: 32GB DDR5 @ 5200 MHz
- L1 Instruction Cache 32.0 KB x 6
- L1 Data Cache 32.0 KB x 6
- L2 Cache 1.00 MB x 6
- L3 Cache 32.0 MB
- Sistema operativo: Arch Linux

3. Desarrollo de la implementación

Esta sección describe cómo se construyó el simulador de tráfico unidimensional con paralelismo OpenMP, explicando las decisiones de diseño y sus consecuencias en rendimiento. La idea central fue mantener una única base de código para la lógica de simulación y variar únicamente el mecanismo de ejecución y el nivel de optimización, de modo que cualquier diferencia de rendimiento sea atribuible al paralelismo o a la estrategia de compilación, y no a cambios en el algoritmo.

3.1. Modelo del autómatas celular de tráfico

El sistema modela una carretera circular de N celdas. Cada celda $i \in \{0, 1, \dots, N - 1\}$ puede estar vacía ($R = 0$) o contener un carro ($R = 1$). En cada paso de tiempo, un carro avanza a la celda siguiente si está libre, o se queda donde está si está ocupada. La regla de actualización es:

$$R(t + 1, i) = [R(t, i) \wedge R(t, (i + 1) \bmod N)] \vee [R(t, (i - 1) \bmod N) \wedge \neg R(t, i)]. \quad (6)$$

El primer término representa un carro que no puede avanzar porque la celda siguiente está ocupada. El segundo representa un carro de la celda anterior que avanza hacia una celda libre. La condición $\neg R(t, i)$ en el segundo término evita que dos carros ocupen la misma celda simultáneamente.

En la implementación base, cada celda se almacena como un entero de 32 bits (`int`). La regla anterior se traduce directamente a operaciones de bits:

$$\text{next}[i] = (\text{here} \& \text{ahead}) \mid (\text{behind} \& \sim \text{here} \& 1), \quad (7)$$

donde `& 1` garantiza que el resultado sea siempre 0 o 1, ya que el operador `~` sobre enteros de 32 bits con signo produce `0xFFFFFFFF` en lugar de 1, lo que corrompería el conteo de carros.

Las lecturas del paso t provienen del arreglo `cells` y las escrituras del paso $t + 1$ van al arreglo `next_cells`. Como son arreglos distintos y cada posición nueva depende únicamente de valores anteriores, todas las iteraciones del bucle son completamente independientes entre sí.

3.2. Velocidad media y estado estacionario

La velocidad media en el paso t es la fracción de carros que se desplazaron:

$$v(t) = \frac{1}{N_c} \sum_{i=0}^{N-1} \mathbf{1}[R(t, i) = 1 \wedge R(t + 1, i) = 0], \quad (8)$$

donde N_c es el número total de carros, que se conserva: la regla (6) nunca crea ni elimina carros. Esto se verifica explícitamente en los tests de corrección.

Partiendo de una distribución inicial aleatoria, el sistema pasa por una fase transitoria donde $v(t)$ fluctúa mientras los carros se reorganizan. Tras suficientes pasos, $v(t)$ se estabiliza en la velocidad asintótica v_∞ , que es el valor de interés. El benchmark reporta el promedio sobre la fase de medición:

$$v_\infty \approx \bar{v} = \frac{1}{T_m} \sum_{t=T_w+1}^{T_w+T_m} v(t), \quad (9)$$

donde T_w es el número de pasos de calentamiento y T_m el número de pasos de medición.

3.3. Criterio de convergencia del calentamiento

La duración de la fase transitoria varía con la densidad y con N . Un número fijo de pasos de calentamiento puede ser insuficiente para densidades cercanas al punto de transición ($\rho \approx 0,5$), o innecesariamente largo para densidades extremas donde el sistema converge rápido.

Para detectar la convergencia de forma automática, se compara la velocidad media en dos ventanas consecutivas de W pasos. El calentamiento termina cuando:

$$\left| \bar{v}_{[t-W, t]} - \bar{v}_{[t-2W, t-W]} \right| < \varepsilon, \quad (10)$$

con $W = 50$, $\varepsilon = 10^{-4}$ y un mínimo de 200 pasos antes de evaluar el criterio, para evitar falsos positivos al inicio. El parámetro `max_warmup_steps` actúa como límite máximo; si la condición (10) no se cumple antes de ese límite, el calentamiento termina de todas formas. En el script de benchmark este techo se fija como $\max(N/10, 2000)$, lo que garantiza al menos el tiempo suficiente para que una perturbación recorra la carretera completa. El número de pasos ejecutados se reporta en `stderr` como diagnóstico.

3.4. Arquitectura de datos

La estructura central es `Road`, que contiene el estado completo de la simulación:

- `cells`: arreglo del estado actual, solo para lectura dentro de un paso.
- `next_cells`: arreglo del siguiente estado, solo para escritura.
- `length`: número de celdas N .
- `car_count`: número de carros N_c , constante durante toda la simulación.

Ambos arreglos se reservan una sola vez en `road_create` y no se realizan asignaciones de memoria dentro del bucle de simulación, lo que elimina ese ruido de las mediciones de tiempo. Al finalizar cada paso, los punteros `cells` y `next_cells` se intercambian directamente sin copiar datos.

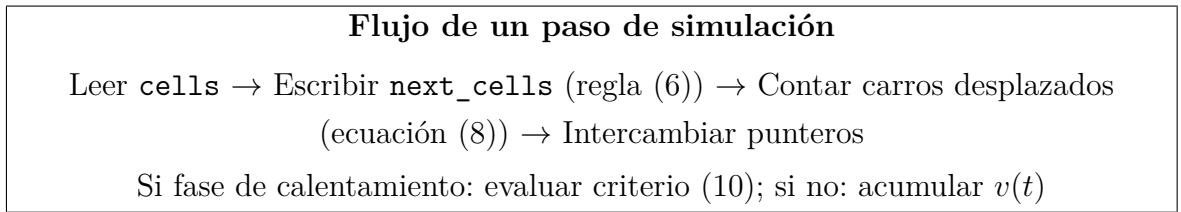


Figura 1: Ciclo compartido por todas las implementaciones.

El estado inicial se genera con `rand` usando como semilla `time(NULL)`. No se controla la semilla explícitamente porque la velocidad asintótica v_∞ es estadísticamente estable respecto a la distribución inicial, como confirman las varianzas de `avg_velocity` entre repeticiones en el CSV de resultados.

3.5. Implementación secuencial

El binario `traffic_seq` se compila sin `-fopenmp`. Las directivas `#pragma omp` presentes en `simulation.c` son ignoradas por el compilador, el binario no crea hilos en tiempo de ejecución y el bucle de actualización se ejecuta de forma ordinaria. Esto puede verificarse con `ldd bin/traffic_seq`, que no lista `libgomp.so`.

La medición de tiempo usa `clock_gettime(CLOCK_MONOTONIC)`, un reloj que no se ve afectado por ajustes del sistema como NTP y garantiza mediciones fiables del tiempo transcurrido. El cronómetro se activa justo antes de la fase de medición y se detiene al terminar, dejando el calentamiento fuera del tiempo reportado.

La interfaz de línea de comandos es:

$$\text{traffic_seq } N \ \rho \ \text{max_warmup } T_m$$

La salida contiene exclusivamente dos valores: el tiempo en milisegundos y la velocidad media $\bar{v}$. Esta separación permite al script de benchmark detectar ejecuciones que no alcanzaron el estado estacionario: una alta varianza en $\bar{v}$ entre repeticiones indica que el techo de calentamiento fue insuficiente.

3.6. Implementación paralela con OpenMP

El binario `traffic_omp` se compila con `-fopenmp`. El número de hilos se lee como argumento de línea de comandos y se comunica al runtime con `omp_set_num_threads`, lo que permite usar un único binario para todos los experimentos sin recompilar.

La interfaz extiende la del binario secuencial con un argumento adicional:

$$\text{traffic_omp } N \ \rho \ \text{max_warmup } T_m \ p$$

donde p es el número de hilos. El valor efectivo se confirma con `omp_get_max_threads()` y se registra en `stderr`.

Por paso de tiempo hay dos regiones paralelas. La primera, en `compute_next_generation`, aplica la regla (6) a cada celda:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < length; i++) { ... }
```

La cláusula `schedule(static)` divide las celdas en bloques contiguos de tamaño fijo, uno por hilo. Esto es adecuado porque el trabajo por celda es uniforme, exactamente tres lecturas, dos operaciones lógicas y una escritura, y produce un patrón de acceso a memoria favorable para el procesador.

La independencia entre iteraciones es completa: todas las lecturas vienen de `current[]` y todas las escrituras van a `next[]`, que son arreglos distintos. No hay condiciones de carrera.

La segunda región paralela, en `count_departed_cars`, cuenta los carros desplazados:

```
#pragma omp parallel for reduction(+:departed) schedule(static)
for (int i = 0; i < length; i++)
    departed += (old_cells[i] == 1 && new_cells[i] == 0) ? 1 : 0;
```

La cláusula `reduction(+:departed)` hace que cada hilo lleve su propio contador local y los sume al final, evitando que los hilos se pisen entre sí. Entre las dos regiones paralelas no se necesita sincronización adicional: OpenMP garantiza automáticamente que todos los hilos terminaron la primera región antes de que cualquiera comience la segunda.

3.7. Optimización por compilador

Para cuantificar el efecto del compilador, se construyen dos variantes adicionales, `traffic_seq_opt` y `traffic_omp_opt`, a partir de los mismos archivos fuente, variando únicamente los flags de compilación.

El bucle principal realiza tres cargas de memoria, dos operaciones lógicas y una escritura por celda, lo que lo convierte en un kernel limitado principalmente por el ancho de banda de memoria. Las flags seleccionadas actúan sobre este perfil:

- `-O3`: activa vectorización automática, transformaciones de bucle e inlining agresivo.
- `-march=native`: emite instrucciones AVX2 o AVX-512 según el procesador. Con datos `int` de 32 bits, un registro AVX2 procesa 8 celdas a la vez; con `uint8_t` procesa 32.
- `-funroll-loops`: reduce el overhead del contador del bucle; más útil cuando los datos caben en L1 ($N \leq 64000$).
- `-flto` con `-fno-semantic-interposition`: permite al compilador fusionar e inlinar funciones entre archivos fuente distintos, eliminando el costo de llamada en cada uno de los $T_m = 1000$ pasos de medición.
- `-falign-loops=32`: alinea el inicio de cada bucle a 32 bytes, evitando que las instrucciones AVX2 crucen fronteras de línea de caché.

Las versiones base se compilan con `-O0` para servir como línea de referencia sin ninguna optimización.

3.8. Optimización de memoria

El profiling reveló que el cuello de botella de la implementación base es la cantidad de instrucciones escalares. El IPC medido es aproximadamente 3.7, cercano al límite del procesador, lo que indica que el CPU ejecuta instrucciones eficientemente sin esperar por datos: el prefetcher secuencial anticipa los accesos y los resuelve mayoritariamente en L1, como confirma el perfil de `perf mem` con una tasa de aciertos en L1 superior al 56 %. El problema no es que falten datos, sino que cada instrucción hace muy poco trabajo útil: con celdas de tipo `int` de 32 bits, un registro AVX2 de 256 bits contiene solo 8 celdas, y el compilador no puede vectorizar eficientemente un bucle cuya lógica opera sobre enteros de ese ancho cuando la semántica real es binaria.

Para explotar esta oportunidad se implementaron `traffic_seq_mem` y `traffic_omp_mem`, que aplican tres optimizaciones de forma conjunta.

3.8.1. MEM-1: Reducción del tipo de celda a `uint8_t`

La estructura `RoadOpt` usa `uint8_t *` en lugar de `int *`. La lógica es idéntica pero se agrega un cast explícito en el complemento:

$$\text{next}[i] = (h \& a) \mid (b \& (\text{uint8_t})(\sim h) \& 1u), \quad (11)$$

para evitar que la extensión de signo corrompa el resultado al escribir en un campo de 8 bits. El efecto inmediato es que un registro AVX2 de 256 bits pasa a contener 32 celdas en lugar de 8, cuadruplicando el trabajo útil por instrucción SIMD. Como consecuencia secundaria, el working set se reduce por un factor de 4, lo que beneficia los tamaños de problema donde los datos con `int` superaban la capacidad de L3, como se resume en la Tabla 1.

Cuadro 1: Working set con `int` vs `uint8_t` para dos buffers en Ryzen 5 7600X.

N	<code>int</code> (bytes)	<code>uint8_t</code> (bytes)	Nivel <code>int</code>	Nivel <code>uint8_t</code>
8 000	64 KB	16 KB	L2	L1
64 000	512 KB	128 KB	L2	L2
500 000	4 MB	1 MB	L3	L2
2 000 000	16 MB	4 MB	L3	L3
5 000 000	40 MB	10 MB	DRAM	L3
10 000 000	80 MB	20 MB	DRAM	L3

Para los tamaños donde ambas representaciones residen en L3 o superiores, la ganancia

proviene casi exclusivamente de la densidad SIMD. Para $N \geq 5\,000\,000$, donde `int` desborda L3 hacia DRAM, la reducción del working set aporta un factor adicional al eliminar accesos a memoria principal.

3.8.2. MEM-2: Fusión de pasadas y eliminación del módulo

La implementación base recorre el arreglo dos veces por paso: una para calcular el siguiente estado y otra para contar los carros desplazados. La solución es fusionar ambas pasadas en una sola función `compute_next_and_count`, que lee y escribe cada celda una sola vez, reduciendo a la mitad el número de recorridos sobre los datos:

```
#pragma omp parallel for schedule(static) reduction(+:departed)
for (int i = 1; i < length - 1; i++) {
    uint8_t b = current[i - 1];
    uint8_t h = current[i];
    uint8_t a = current[i + 1];
    uint8_t n = (h & a) | (b & (uint8_t)(~h) & 1u);
    next[i]    = n;
    departed += (h == 1u && n == 0u) ? 1 : 0;
}
```

Las celdas de los extremos ($i = 0$ e $i = N - 1$), que requieren índices circulares, se calculan fuera del bucle paralelo. Esto elimina el operador módulo del camino crítico y deja el bucle interno libre de bifurcaciones adicionales, condición necesaria para que el compilador pueda vectorizarlo con instrucciones AVX2.

3.8.3. MEM-3: Alineación de buffers a línea de caché

Los arreglos se reservan con `posix_memalign(64, ...)` en lugar de `malloc`, garantizando que el inicio de cada arreglo coincida con el inicio de una cacheline de 64 bytes. Esto tiene dos efectos: los accesos en los extremos del rango de cada hilo no afectan la línea de caché de otro hilo, evitando false sharing, y el prefetcher puede anticipar los accesos más eficientemente desde una dirección base alineada.

3.9. Matriz de variantes de compilación

Las cuatro estrategias anteriores producen ocho binarios:

Cuadro 2: Binarios generados por `make all` y sus características.

Binario	impl en CSV	OMP	OPT flags	Mem opt
<code>traffic_seq</code>	<code>traffic_seq</code>	No	No	No
<code>traffic_seq_opt</code>	<code>traffic_seq_opt</code>	No	Sí	No
<code>traffic_omp</code>	<code>traffic_omp</code>	Sí	No	No
<code>traffic_omp_opt</code>	<code>traffic_omp_opt</code>	Sí	Sí	No
<code>traffic_seq_mem</code>	<code>traffic_seq_mem</code>	No	No	Sí
<code>traffic_seq_mem_opt</code>	<code>traffic_seq_mem_opt</code>	No	Sí	Sí
<code>traffic_omp_mem</code>	<code>traffic_omp_mem</code>	Sí	No	Sí
<code>traffic_omp_mem_opt</code>	<code>traffic_omp_mem_opt</code>	Sí	Sí	Sí

3.10. Diseño del script de pruebas

El script `bench_traffic.sh` actúa como harness de medición: su única responsabilidad es invocar los binarios con los parámetros correctos, capturar su salida y guardarla en un CSV.

3.11. Matriz de configuraciones

El harness define dos experimentos que cubren las preguntas de rendimiento centrales del trabajo.

El experimento de escalado mide todas las combinaciones de los seis valores de N con los seis conteos de hilos $\{0, 2, 4, 6, 8, 12\}$ a densidad fija $\rho = 0,50$, para los ocho binarios de la Tabla 2. El valor $p = 0$ indica los binarios secuenciales; cualquier $p > 0$ invoca los binarios OpenMP con ese número de hilos. Los valores de N cubren cada nivel de la jerarquía de memoria con la representación base; la Tabla 1 muestra cómo `uint8_t` desplaza varios tamaños hacia niveles superiores de caché.

3.12. Techo de calentamiento adaptativo

El techo de pasos de calentamiento que el harness pasa a cada binario se calcula como:

$$T_{w,\text{máx}} = \text{máx}\left(\frac{N}{10}, 2000\right). \quad (12)$$

El factor $N/10$ garantiza que el techo sea proporcional al tiempo que tarda una perturbación en recorrer la carretera, con un margen de seguridad de un orden de magnitud. El piso de 2000 evita techos demasiado bajos para valores pequeños de N . El binario puede terminar

antes si el criterio (10) se cumple, lo que ocurre en la práctica para densidades extremas ($\rho \leq 0,1$ o $\rho \geq 0,9$) en pocas centenas de pasos.

3.13. Formato del CSV de resultados

El archivo `data.csv` tiene siete columnas:

Columna	Descripción
<code>impl</code>	Binario usado (véase Tabla 2)
<code>threads</code>	Número de hilos (0 para variantes secuenciales)
<code>road_length</code>	Tamaño de la carretera N
<code>density</code>	Densidad inicial de carros ρ
<code>repetition</code>	Índice de repetición (1 a 10)
<code>wall_time_ms</code>	Tiempo de la fase de medición en ms
<code>avg_velocity</code>	Velocidad media $\bar{v}$ sobre los T_m pasos

4. Perfilado de código

El perfilado de código (*profiling*) es el proceso de medir el comportamiento de un programa en ejecución con el objetivo de identificar dónde se concentra el costo computacional y de memoria. A diferencia del análisis estático, el perfilado opera sobre la ejecución real del programa, capturando métricas como tiempo por función, contadores de hardware del procesador, comportamiento de la jerarquía de caché y consumo dinámico de memoria. Esto permite construir evidencia empírica que justifica las decisiones de optimización y sirve de línea base para comparar implementaciones alternativas.

El script `bench_profiling.sh` orquesta cinco técnicas de perfilado complementarias sobre la implementación secuencial del autómata celular de tráfico, para los tamaños de carretera $N \in \{8000, 64000, 500000, 2000000\}$ con densidad fija de 0.50 y 1000 pasos de simulación por medición. Cada técnica responde una pregunta distinta: cuánto tiempo tarda, en qué función se acumula ese tiempo, qué eventos de hardware ocurren, qué nivel de caché sirve los accesos, y cuánta memoria dinámica se consume. Los resultados se consolidan en un archivo CSV (`data_profiling.csv`) que sirve de entrada al generador de reportes.

4.1. Binarios compilados

El script genera dos binarios distintos a partir de las mismas fuentes (`road.c`, `simulation.c`, `cli_args.c`, `timing.c`, `seq_main.c`):

- El binario `traffic_seq_perf` se compila con `gcc -g -std=c11 -D_POSIX_C_SOURCE=200809L`, sin optimizaciones de nivel, activando símbolos de depuración (`-g`) para que `perf` y Valgrind puedan asociar las muestras con el código fuente. Este binario es el que utilizan `perf stat`, `perf record`, Valgrind Massif y Valgrind Cachegrind.
- El binario `traffic_seq_gprof` se compila adicionalmente con `-fno-inline -pg`. La flag `-pg` instruye al compilador para insertar llamadas a `mcount()` al inicio de cada función, generando el archivo `gmon.out` durante la ejecución. La flag `-fno-inline` impide que el compilador funda el cuerpo de funciones pequeñas en sus sitios de llamada, garantizando que `gprof` pueda contabilizar cada función de forma independiente.

4.2. gprof: perfilado por instrumentación del compilador

`gprof` es una herramienta de perfilado basada en instrumentación generada en tiempo de compilación. El script ejecuta `traffic_seq_gprof` con los mismos parámetros de calentamiento y medición, lo que genera el archivo `gmon.out`, y luego invoca `gprof` para producir el reporte textual.

El reporte tiene dos partes principales. El *flat profile* lista cada función con el porcentaje del tiempo total que le corresponde, los segundos acumulados, el número de invocaciones y el tiempo promedio por llamada; responde directamente la pregunta de en qué función pasa el programa la mayor parte del tiempo. El *call graph* muestra la jerarquía de llamadas y cuánto tiempo se propaga desde cada función hija hacia su función padre, lo que permite distinguir si una función es costosa por su propia lógica o por las subfunciones que invoca.

La limitación principal de `gprof` es que su medición del tiempo se basa en muestreo estadístico del contador de programa con una resolución aproximada de 10 ms, lo que introduce ruido estadístico para funciones de corta duración. Además, el binario instrumentado con `-pg` no es idéntico al binario de producción, lo que puede alterar el comportamiento de caché y el layout de código.

4.3. perf stat: contadores de hardware de la PMU

`perf stat` accede directamente a la *Performance Monitoring Unit* (PMU), un conjunto de registros físicos del procesador diseñados para contar eventos de hardware con overhead mínimo sobre el programa medido. El script lo ejecuta con `-r 5` (`PERF_STAT_REPEATS=5`) para promediar los contadores a través de cinco repeticiones y reducir el ruido estadístico de la PMU.

Los eventos medidos cubren cuatro categorías de cuello de botella:

Throughput del pipeline. Los eventos `cycles` e `instructions` permiten calcular el IPC:

$$\text{IPC} = \frac{\text{instructions}}{\text{cycles}}. \quad (13)$$

En el autómata celular, donde el bucle de actualización es simple y altamente regular, un IPC bajo señalaría stalls del pipeline por esperas de memoria, no por complejidad del flujo de control.

Caché de último nivel (LLC). Los eventos `cache-references` y `cache-misses` miden la presión sobre la LLC. La tasa de fallo

$$\text{LLC miss rate} = \frac{\text{cache-misses}}{\text{cache-references}} \times 100 \% \quad (14)$$

indica qué fracción de los accesos a la LLC no encontró el dato y debió recurrir a DRAM. Para tamaños grandes de N la carretera completa y los buffers auxiliares pueden no caber en caché, haciendo que esta tasa sea el principal indicador de degradación.

Caché L1 de datos (L1-D) y TLB. Los eventos `L1-dcache-loads` y `L1-dcache-load-misses` miden la localidad efectiva de acceso a datos en el nivel de caché más próximo al procesador. Los eventos `dTLB-loads` y `dTLB-load-misses` miden el costo de traducción de direcciones virtuales a físicas, relevante cuando el array de la carretera se distribuye en muchas páginas de memoria virtual para los valores grandes de N .

Predictor de saltos. Los eventos `branch-instructions` y `branch-misses` cuantifican la efectividad del predictor. Dado que el autómata celular recorre el array con un bucle perfectamente regular y las condiciones de actualización dependen de los valores de las celdas vecinas, se espera una tasa de fallo moderada pero estable, distinta a la que exhibiría un algoritmo con flujo de control irregular.

Un detalle técnico del script es la función `perf_extract`, que extrae los contadores del reporte de texto mediante una expresión regular anclada en la frontera de palabra para evitar que el término `instructions` colisione con `branch-instructions` al realizar el `grep`.

4.4. `perf record`: muestreo basado en interrupciones de hardware

Mientras que `perf stat` reporta totales globales, `perf record` ofrece una vista espacial de dónde en el código ocurren los eventos de CPU. El script lo ejecuta con `-F 999` para una

frecuencia de muestreo de aproximadamente 999 interrupciones por segundo, y con `-g` para capturar el call stack completo en el momento de cada interrupción.

El reporte se genera con `perf report -stdio -no-children`, lo que produce un histograma que responde la pregunta de qué fracción del tiempo de CPU estuvo el procesador ejecutando instrucciones de cada función. A diferencia de `gprof`, este perfilador no requiere recompilar el programa y mide el binario en condiciones reales de ejecución, incluyendo el código de bibliotecas del sistema. El archivo `perf.data` intermedio se elimina tras generar el reporte para no ocupar espacio en disco.

4.5. Valgrind Massif: perfilado de uso del heap

Massif es una herramienta del framework Valgrind que ejecuta el programa dentro de una máquina virtual de instrumentación binaria e intercepta todas las llamadas a las funciones de gestión de memoria dinámica (`malloc`, `calloc`, `realloc`, `free`) sin modificar el código fuente. Dada la penalización en tiempo de ejecución que impone Valgrind, el script limita esta técnica a los tamaños $N \leq 500000$; para $N = 2000000$ se escribe el centinela `SKIPPED` en el archivo de reporte para que la función de extracción no confunda el salto intencional con un fallo de ejecución.

El valor extraído y almacenado en el CSV es el pico de heap en MB, obtenido leyendo la columna `total(B)` del reporte de `ms_print`, que incluye tanto los bytes útiles como el overhead del gestor de memoria. Para el autómata celular, donde se reservan dinámicamente dos arrays de enteros de tamaño N (la carretera en el instante t y la copia en $t+1$), el modelo teórico del pico es:

$$M_{\text{teórico}} = \frac{2 \cdot N \cdot 4}{1024^2} \text{ MB.} \quad (15)$$

La diferencia entre el valor medido y este modelo cuantifica el overhead de metadata y alineación del asignador de memoria del sistema.

4.6. Valgrind Cachegrind: simulación de la jerarquía de caché

Cachegrind simula la jerarquía de caché del procesador mediante instrumentación a nivel de instrucción, interceptando cada operación de carga y almacenamiento y procesándola a través de un modelo de caché configurable. A diferencia de `perf stat`, no utiliza contadores de hardware reales, lo que hace sus resultados completamente determinísticos y reproducibles entre ejecuciones. Aplica la misma restricción de tamaño que Massif: se ejecuta solo para $N \leq 500000$.

La métrica fundamental que produce Cachegrind es `Ir` (*Instruction References*): el con-

teo exacto de instrucciones ejecutadas durante toda la corrida. El reporte generado por `cg_annotate` presenta tres niveles de granularidad: totales del programa, distribución porcentual por función y distribución línea a línea dentro del código fuente. Esta vista anotada permite identificar con precisión cuáles sentencias del bucle de actualización del autómata son las más costosas en términos de instrucciones ejecutadas.

4.7. Visión de conjunto

Las cinco técnicas de perfilado cubren dimensiones distintas del comportamiento del programa, y ninguna por sí sola es suficiente para construir un diagnóstico completo. La Tabla 3 resume su complementariedad.

Cuadro 3: Herramientas de perfilado utilizadas, su tipo de medición y su limitación principal

Herramienta	Tipo	Pregunta que responde	Limitación principal
Temporización	Wall-clock estático	¿Cuánto tarda? ¿Es estable?	Sin resolución por función
<code>gprof</code>	Instrumentación compilador	¿Qué función consume más CPU?	Binario modificado; resolución de 10 ms
<code>perf stat</code>	Contadores hardware (PMU)	¿Qué eventos de hardware ocurren?	Solo totales globales
<code>perf record</code>	Muestreo por interrupciones	¿Dónde está el CPU la mayor parte del tiempo?	Resolución de ~ 1 ms
Valgrind Massif	Intercepción de heap	¿Cuánta memoria dinámica se usa?	$5\text{--}30\times$ overhead; solo $N \leq 500000$
Valgrind Cachegrind	Simulación de caché	¿Qué instrucciones son más costosas?	Modelo aproximado; $20\text{--}100\times$ overhead; solo $N \leq 500000$

4.8. Resultados del perfilado

La Tabla 4 consolida los resultados de las cinco técnicas de perfilado aplicadas a la implementación secuencial del autómata celular para los tamaños de carretera $N \in \{8000, 64000, 500000, 2000000\}$ con densidad fija $\rho = 0,50$ y 1000 pasos de simulación

por medición. El uso de distintos tamaños permite caracterizar cómo escala el comportamiento del programa a medida que el working set supera los sucesivos niveles de la jerarquía de memoria, mientras que el análisis del hotspot se centra en $N = 2000000$ por ser el caso más representativo del comportamiento en producción.

Cuadro 4: Caracterización de la implementación secuencial del autómatas celular de tráfico

N	$\bar{t}$ (ms)	σ (ms)	CV (%)	Mcells/s	IPC	L1 miss %	LLC miss %	dTLB miss %	Heap (MB)
8 000	31.111	0.113	0.36	257.14	3.7164	1.33	0.00	7.49	0.061
64 000	249.038	0.441	0.18	256.99	3.7484	1.34	0.00	0.08	0.488
500 000	1953.388	1.720	0.09	255.97	3.7072	1.34	0.00	0.30	3.815
2 000 000	7874.508	28.544	0.36	253.98	3.6786	1.33	0.00	1.41	—

4.8.1. Localización del hotspot

Las tres herramientas de perfilado funcional convergen en señalar `compute_next_generation` como la única fuente relevante de cómputo. Para $N = 2000000$, gprof le atribuye el 71.52 % del tiempo muestral, mientras que perf record confirma este resultado con el 71.97 % de los ciclos observados; la diferencia de apenas 0.45 puntos porcentuales entre ambas herramientas valida la coherencia de la medición a pesar de que operan por principios distintos (instrumentación por compilación frente a muestreo por interrupciones de hardware). El 28.48 % restante corresponde a `count_departed_cars`, y las funciones de infraestructura (`simulation_step`, `swap_generation_buffers`, `allocate_int_array`) acumulan una fracción insignificante.

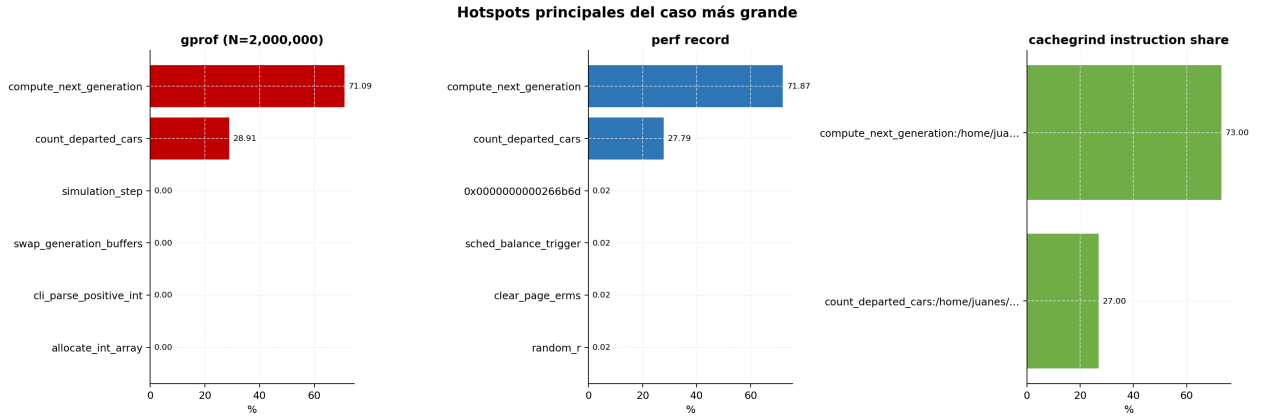


Figura 2: Distribución del tiempo de CPU por función según gprof y perf record para $N = 2000000$

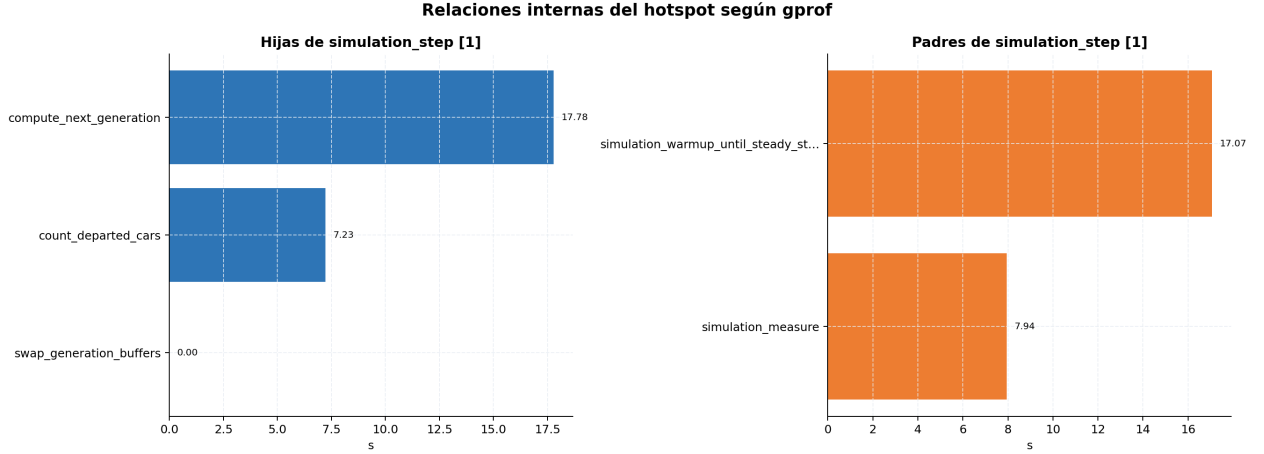


Figura 3: Hostspot interno de `compute_next_generation` según gprof para $N = 2000000$

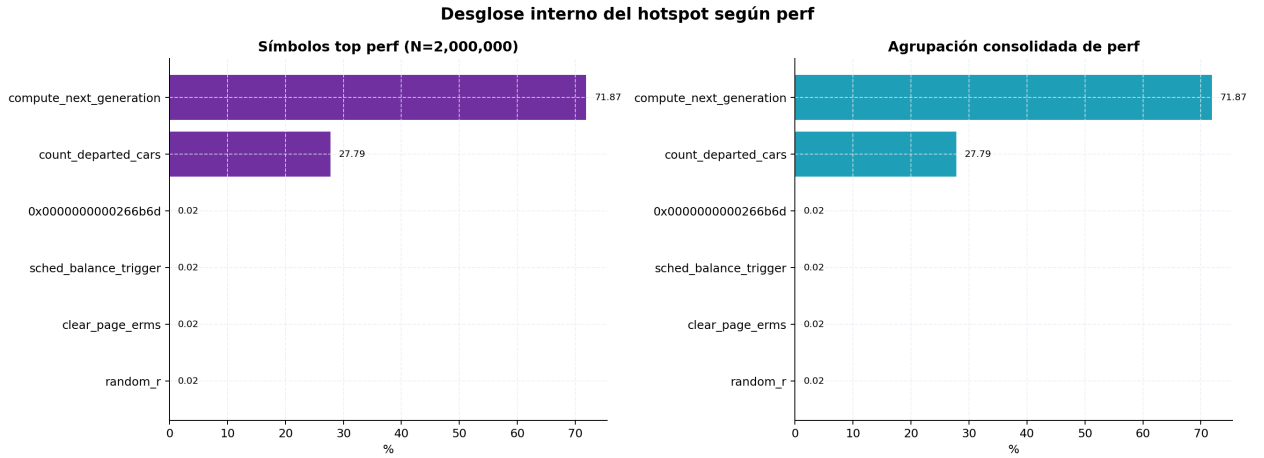


Figura 4: Hostspot interno de `compute_next_generation` según perf para $N = 2000000$

Valgrind Cachegrind refuerza este diagnóstico con mayor resolución: `compute_next_generation` concentra el 72.9 % de las referencias de instrucción para $N = 8000$, el 73 % para $N = 64000$ y el 73 % para $N = 500000$, con una estabilidad notable entre tamaños que confirma que la fracción de cómputo atribuible al bucle de actualización no depende del tamaño del problema. El análisis de línea anotada identifica dos instrucciones dominantes dentro de esa función: el cálculo del índice periódico `int cell_behind = current[(i-1+length) % length];` acumula el 19% de todas las referencias de instrucción del programa, y la detección de coches que avanzan `departed += (old_cells[i]==1 && new_cells[i]==0) ? 1 : 0;` acumula el 21.1 %. Ambas líneas son el objetivo natural de cualquier optimización dirigida a esta función.

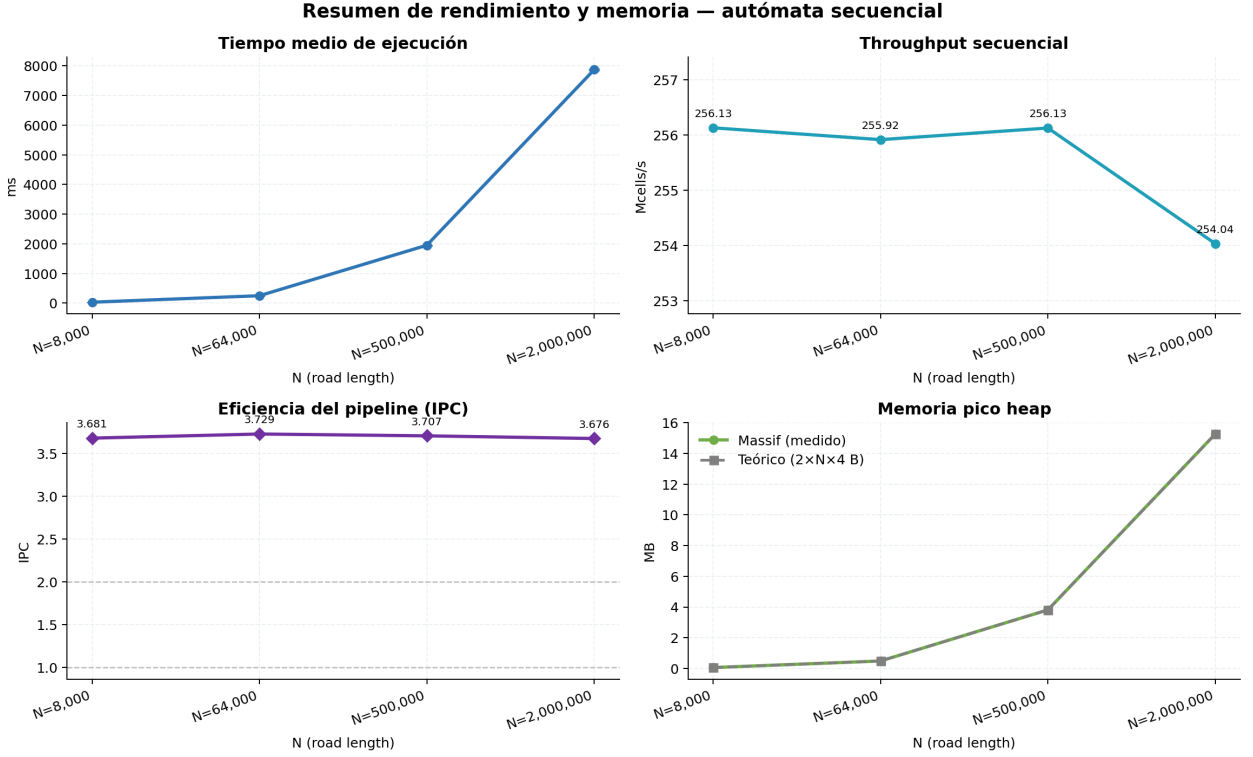


Figura 5: Resumen de métricas de hardware

El análisis con Valgrind Massif muestra que el consumo de heap escala linealmente con N , pasando de 0.061 MB para $N = 8000$ hasta 3.815 MB para $N = 500000$. Este comportamiento coincide con el modelo teórico $M_{\text{teórico}} = 2 \cdot N \cdot 4/1024^2$ MB derivado de los dos arrays de enteros de tamaño N reservados dinámicamente: la relación entre el valor medido y el teórico es del 99.94 % para $N = 8000$ y $N = 64000$, y del 100.01 % para $N = 500000$. La fracción de bytes útiles respecto al total medido supera el 99.95 % en todos los tamaños, lo que confirma que el gestor de memoria del sistema no introduce overhead de fragmentación ni metadata anómala y que el consumo observado se explica íntegramente por las reservas explícitas del programa. Para $N = 2000000$ el valor teórico esperado es de aproximadamente 15.26 MB, pero Massif no pudo ejecutarse por superar el límite $N_{\text{máx}} = 500000$ establecido en el script.

4.8.2. Estabilidad temporal y throughput

El throughput se mantiene prácticamente constante en torno a 254–256 Mcells/s a lo largo de todos los tamaños, con una caída máxima del 1.2 % al pasar de $N = 8000$ a $N = 2000000$. Esta estabilidad refleja que el núcleo de cómputo del autómata es aritméticamente simple (dos operaciones lógicas y tres lecturas por celda) y el hardware puede sostener una tasa de actualización prácticamente uniforme con independencia de la longitud de la carretera para los tamaños evaluados.

La velocidad media de los coches converge a $\bar{v} \approx 0,99$ para todos los tamaños con $\rho = 0,50$, lo que confirma que el simulador alcanza el estado estacionario antes de que comiencen las mediciones de temporización. Para una densidad de 0.50, el modelo teórico predice velocidad próxima a 1.0 porque aproximadamente la mitad de las celdas están vacías y la mayoría de los coches dispone de espacio libre adelante; la coherencia entre esta predicción y los valores medidos valida tanto la corrección del algoritmo como el procedimiento de calentamiento.

4.8.3. Limitaciones de hardware

Los contadores de la PMU revelan un perfil de hardware favorable que contrasta con el de algoritmos de mayor densidad aritmética. El IPC se mantiene estable entre 3.68 y 3.75 en todos los tamaños, lo que indica que el pipeline del procesador retira aproximadamente 3.7 instrucciones por ciclo de forma sostenida. Este valor alto es coherente con la naturaleza del bucle: las instrucciones son simples (cargas, operaciones bit a bit, saltos predecibles) y el patrón de acceso a memoria es estrictamente secuencial, lo que permite al hardware prefetcher anticipar las líneas de caché necesarias antes de que el procesador las solicite.

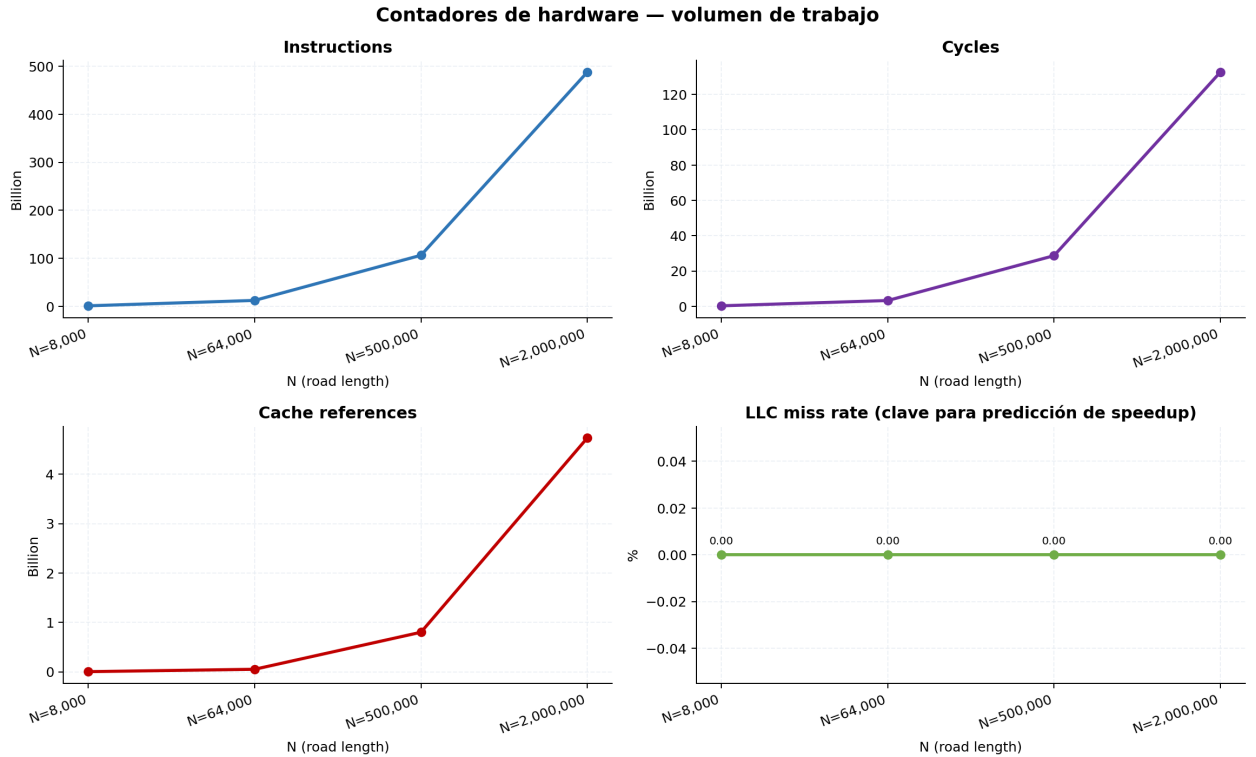


Figura 6: Contadores de hardware

La tasa de fallos en L1 se sitúa en torno al 1.33–1.34% de forma uniforme, muy por debajo del 29.74% observado en la multiplicación de matrices para $N = 1024$ con patrón

de acceso columnar. La tasa de fallos de LLC es exactamente 0.00 % en todos los tamaños medidos, incluyendo $N = 2000000$, cuyo working set teórico es $2 \times 2000000 \times 4 \approx 16$ MB y supera la capacidad típica de L3. Este resultado aparentemente contradictorio se explica por el prefetcher de hardware: el acceso estrictamente secuencial al array de la carretera permite al prefetcher cargar líneas de caché con suficiente anticipación como para que los accesos sean atendidos sin registrar un fallo en LLC, aunque el dato haya sido traído desde DRAM. El cuello de botella para N grandes no es por tanto la latencia de caché sino el ancho de banda del bus de memoria, que no queda directamente expuesto por los contadores de LLC miss.

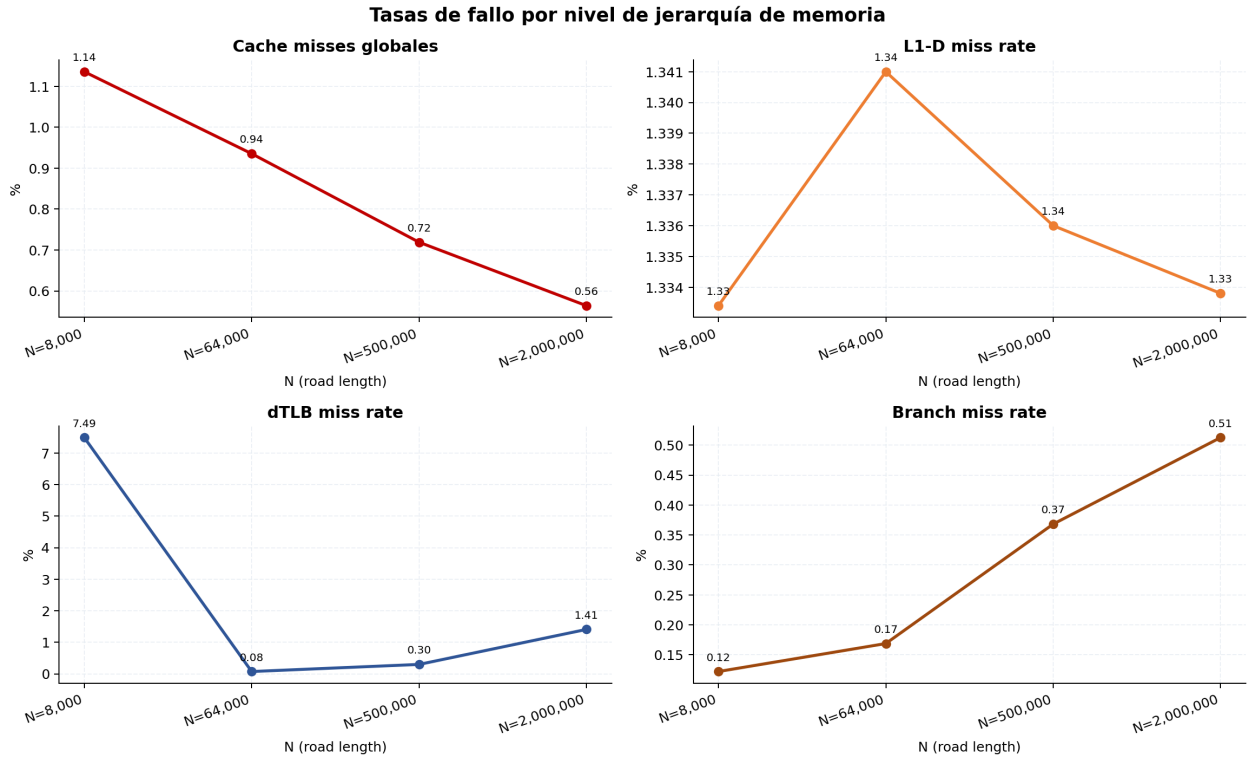


Figura 7: Tasas de fallo de caché y TLB

Los fallos del predictor de ramas se mantienen por debajo del 0.52 % en todos los casos, lo que es consistente con el flujo de control regular del bucle de actualización. El único contador con variabilidad notable entre tamaños es dTLB miss: el valor elevado de 7.49 % para $N = 8000$ y su caída a 0.08 % para $N = 64000$ sugieren que para la carretera más corta el acceso a las celdas mediante índice modular genera un patrón de acceso a páginas virtuales que el TLB aún no tiene precargado al inicio de la medición; al crecer N las páginas se reutilizan con mayor frecuencia y la presión sobre el TLB disminuye.

4.8.4. Accesos a memoria por nivel

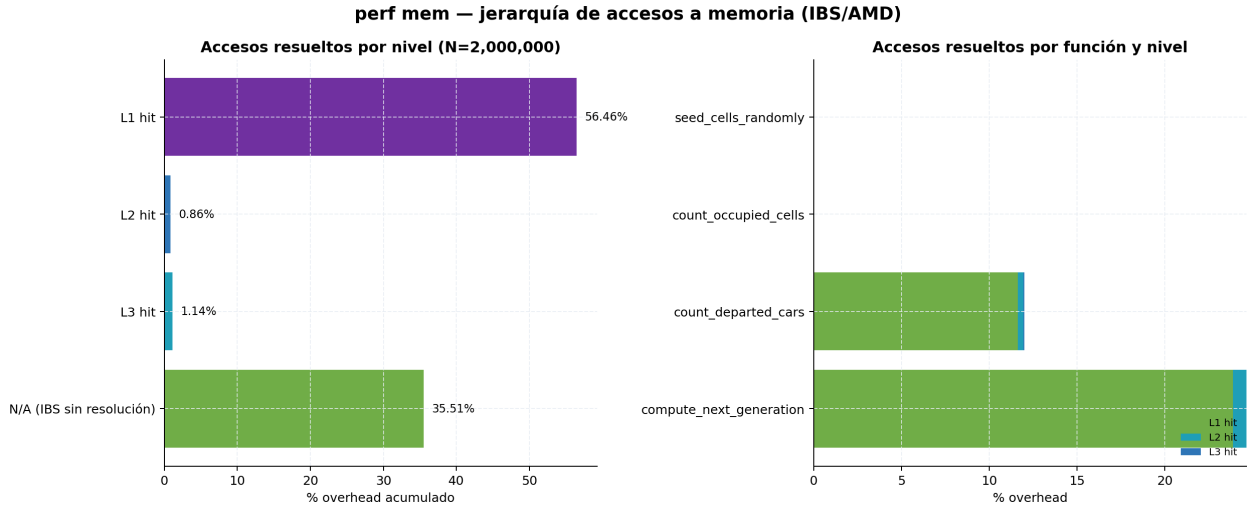


Figura 8: Accesos a memoria por nivel

Viendo los accesos a memoria por nivel, el 56.46 % de las referencias de instrucción se atienden en L1, lo que es coherente con la tasa de fallo de L1 del 1.33 %, la mayoría de los accesos a datos son resueltos por el prefetcher en L1 antes de que el procesador los solicite, apenas el 2 % combinado en L2+L3. El 35.51 % N/A en IBS son muestras que no apuntaron a ningún acceso de memoria, típicamente operaciones sobre registros o instrucciones de cómputo puro.

5. Pruebas

En esta sección se presentan las pruebas realizadas para evaluar el rendimiento de los algoritmos implementados. Se llevaron a cabo experimentos tanto en modo secuencial como en modo paralelo, utilizando diferentes tamaños de entrada y distintas configuraciones de optimización con el objetivo de comparar su impacto sobre el tiempo de ejecución.

Para la optimización por compilador se utilizó el siguiente conjunto de banderas:

- **-O3**: activa el nivel máximo de optimización general del compilador, aplicando transformaciones agresivas sobre el código para mejorar el rendimiento.
- **-march=native**: genera código optimizado específicamente para la arquitectura del procesador en el que se compila, aprovechando sus extensiones e instrucciones disponibles.
- **-funroll-loops**: desborda los ciclos cuando es posible, reduciendo el costo de control del bucle y mejorando la ejecución en regiones críticas.

- `-flto`: habilita la optimización en tiempo de enlace, permitiendo al compilador analizar y optimizar funciones y módulos completos de forma conjunta.
- `-fno-semantic-interposition`: permite al compilador asumir que las llamadas internas no serán interceptadas dinámicamente, lo que facilita optimizaciones más agresivas como inlining y desvirtualización.
- `-falign-loops=32`: alinea los bucles en fronteras de 32 bytes para mejorar el aprovechamiento de la caché de instrucciones y la eficiencia del flujo de ejecución.
- `-fomit-frame-pointer`: elimina el puntero de marco cuando no es necesario, liberando un registro adicional y reduciendo sobrecarga en funciones simples o críticas.

5.1. Resultados de los algoritmos secuenciales

En esta sección se presentan los resultados de las cuatro implementaciones secuenciales evaluadas. La versión *base* corresponde a la implementación directa del algoritmo con arreglos de tipo `int` y sin optimizaciones adicionales; la versión *compiler opt* incorpora optimizaciones de compilación orientadas a mejorar la generación de código; la versión *memory opt* utiliza una representación de memoria más compacta para reducir el consumo y mejorar la localidad de datos; y la versión *compiler + memory opt* combina ambas estrategias con el objetivo de aprovechar simultáneamente mejoras en el cómputo y en el acceso a memoria. A partir de estas variantes se comparan los tiempos de ejecución y se identifica la mejor base secuencial para contrastarla posteriormente con las versiones paralelas.

Serial Base						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	23,843	189,473	1.467,183	5.898,853	14.769,370	29.595,217
2	23,908	189,508	1.462,836	5.900,844	14.781,014	29.622,950
3	23,869	189,343	1.467,137	5.894,656	14.789,987	29.588,806
4	23,621	188,610	1.469,392	5.909,635	14.772,112	29.594,913
5	23,981	189,967	1.468,515	5.911,793	14.796,639	29.632,180
6	23,250	188,597	1.467,143	5.894,014	14.781,607	29.615,555
7	23,902	189,680	1.463,583	5.892,064	14.788,313	29.584,096
8	23,774	189,183	1.463,927	5.891,131	14.803,116	29.615,908
9	23,869	189,619	1.471,247	5.900,426	14.795,011	29.625,131
10	23,794	189,924	1.466,482	5.903,389	14.801,609	29.614,191
Promedio	23,781	189,390	1.466,745	5.899,681	14.787,878	29.608,895
Desv. Est.	0,199	0,454	2,534	6,713	11,093	15,894
CV (%)	0,84	0,24	0,17	0,11	0,08	0,05

Cuadro 5: Tiempos de ejecución del algoritmo secuencial base

Serial + Compiler Opt						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	21,044	169,146	1.325,158	5.308,188	13.561,568	27.164,075
2	21,043	168,823	1.325,458	5.310,818	13.560,901	27.110,681
3	21,050	168,756	1.325,141	5.322,307	13.588,794	27.090,437
4	21,037	168,950	1.324,660	5.311,844	13.568,664	27.106,895
5	21,027	169,063	1.324,793	5.308,325	13.560,719	27.105,838
6	21,065	168,930	1.324,869	5.310,250	13.567,036	27.102,050
7	21,077	169,156	1.325,018	5.310,481	13.567,250	27.119,746
8	21,050	169,151	1.325,708	5.314,236	13.563,924	27.170,153
9	21,043	168,760	1.325,827	5.309,435	13.564,898	27.108,190
10	21,047	168,788	1.325,089	5.311,385	13.565,553	27.103,063
Promedio	21,048	168,952	1.325,172	5.311,727	13.566,931	27.118,113
Desv. Est.	0,013	0,158	0,364	3,902	7,745	25,503
CV (%)	0,06	0,09	0,03	0,07	0,06	0,09

Cuadro 6: Tiempos de ejecución del algoritmo secuencial optimizado por compilador

Serial + Memory Opt						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	23,471	187,230	1.490,743	6.066,968	15.179,209	30.116,255
2	23,800	187,370	1.498,853	6.073,201	15.159,005	30.104,120
3	23,426	188,107	1.498,923	6.088,770	15.187,019	30.286,106
4	23,331	186,841	1.494,814	6.060,789	15.190,887	30.106,231
5	23,516	186,555	1.500,923	6.061,229	15.145,533	30.138,303
6	23,441	186,529	1.500,854	6.043,667	15.128,532	30.116,997
7	23,449	186,422	1.490,155	6.059,249	15.150,494	30.136,636
8	23,483	187,059	1.489,798	6.097,657	15.177,185	30.187,690
9	23,272	186,323	1.495,491	6.069,529	15.182,731	30.116,636
10	23,441	186,689	1.493,633	6.073,848	15.167,515	30.146,248
Promedio	23,463	186,912	1.495,419	6.069,491	15.166,811	30.145,522
Desv. Est.	0,132	0,517	4,107	14,560	19,361	52,380
CV (%)	0,56	0,28	0,27	0,24	0,13	0,17

Cuadro 7: Tiempos de ejecución del algoritmo secuencial con optimización de memoria

Serial + Compiler & Memory Opt						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	0,387	2,754	22,467	91,198	228,620	455,020
2	0,360	2,747	22,433	90,059	226,004	454,266
3	0,359	2,760	22,596	89,730	228,227	454,143
4	0,457	2,748	22,437	90,052	226,365	448,395
5	0,366	2,757	22,618	89,859	225,567	455,128
6	0,365	2,753	22,570	90,094	228,372	452,826
7	0,365	2,761	22,563	89,807	225,392	453,468
8	0,360	2,754	22,512	89,940	228,635	455,560
9	0,362	2,755	22,469	89,870	228,844	454,098
10	0,359	2,750	22,485	90,126	225,122	448,253
Promedio	0,374	2,754	22,515	90,073	227,115	453,116
Desv. Est.	0,029	0,004	0,064	0,395	1,467	2,513
CV (%)	7,69	0,16	0,28	0,44	0,65	0,55

Cuadro 8: Tiempos de ejecución del algoritmo secuencial con optimización de memoria y compilador

Tabla 2 — Promedio y Speedup (ref: Serial Base)													
Implementación	N=8K Avg (ms)	N=8K Speedup	N=64K Avg (ms)	N=64K Speedup	N=500K Avg (ms)	N=500K Speedup	N=2M Avg (ms)	N=2M Speedup	N=5M Avg (ms)	N=5M Speedup	N=10M Avg (ms)	N=10M Speedup	SpUp Prom.
Serial Base	23,781	1,0000	189,390	1,0000	1.466,745	1,0000	5.899,681	1,0000	14.787,878	1,0000	29.608,895	1,0000	1,00
Serial + Compiler Opt	21,048	1,1298	168,952	1,1210	1.325,172	1,1068	5.311,727	1,1107	13.566,931	1,0900	27.118,113	1,0918	1,11
Serial + Memory Opt	23,463	1,0136	186,912	1,0133	1.495,419	0,9808	6.069,491	0,9720	15.166,811	0,9750	30.145,522	0,9822	0,99
Serial + Compiler & Memory Opt	0,374	63,5858	2,754	68,7717	22,515	65,1452	90,073	65,4985	227,115	65,1119	453,116	65,3451	65,58

Cuadro 9: Speedup del algoritmo secuencial base

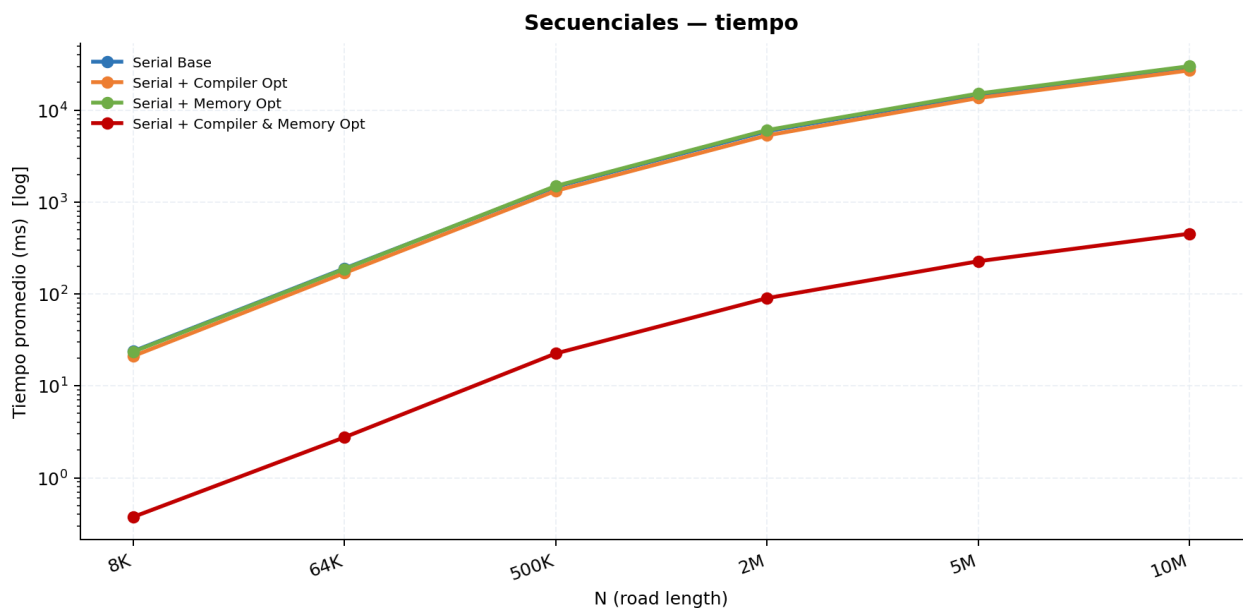


Figura 9: Tiempos de ejecución de los algoritmos secuenciales

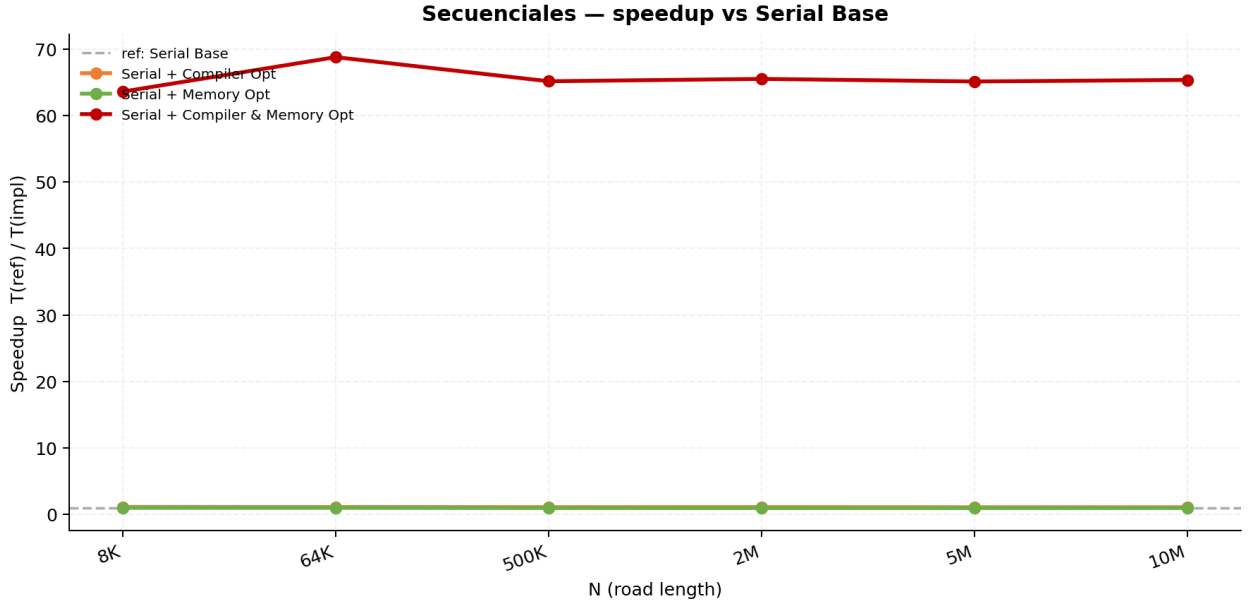


Figura 10: Speedup de los algoritmos secuenciales

Como se puede observar la mejor versión secuencial es la que combina optimización de memoria y optimización por compilador, alcanzando un speedup de aproximadamente 65x en comparación con la versión base, lo cual representa una mejora enorme. Esto se debe a que la optimización de memoria reduce significativamente el tiempo de acceso a datos, mientras que las optimizaciones del compilador mejoran la eficiencia del código generado. En contraste, la versión con solo optimización por compilador o solo optimización de memoria muestra mejoras menores, lo que indica que ambas estrategias son complementarias y su combinación es clave para maximizar el rendimiento en este caso. Teniendo esto en cuenta, se seleccionará la versión *compiler + memory opt* como la base secuencial para compararla posteriormente con las implementaciones paralelas y evaluar el impacto de la paralelización sobre el rendimiento.

5.2. Resultados de la implementación con OpenMP

OpenMP Base (p=2)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	24,930	192,965	1.492,015	5.980,932	14.920,732	29.891,671
2	24,973	192,559	1.493,127	5.973,327	14.922,879	29.921,315
3	25,055	193,710	1.491,527	5.976,337	14.949,657	29.916,840
4	25,034	192,930	1.489,816	5.967,405	14.954,899	29.906,151
5	25,000	192,390	1.492,522	5.980,884	14.963,768	29.900,165
6	25,040	194,011	1.489,379	5.968,237	14.950,966	29.902,177
7	24,823	192,271	1.493,583	5.971,682	14.956,593	29.857,147
8	25,064	192,775	1.489,298	5.983,985	14.946,430	29.843,324
9	25,141	192,783	1.495,324	5.971,146	14.945,228	29.908,973
10	24,943	192,913	1.490,763	5.981,680	14.953,758	29.900,524
Promedio	25,000	192,931	1.491,735	5.975,561	14.946,491	29.894,829
Desv. Est.	0,084	0,518	1,873	5,709	13,329	23,899
CV (%)	0,33	0,27	0,13	0,10	0,09	0,08

Cuadro 10: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 2 hilos

OpenMP Base (p=4)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	13,461	97,275	746,868	2.988,807	7.469,089	15.020,833
2	13,299	97,733	746,081	2.985,748	7.471,532	14.986,478
3	13,271	97,831	746,677	2.982,932	7.472,163	14.994,708
4	13,378	97,344	748,167	2.979,351	7.476,191	15.130,795
5	13,285	97,461	748,139	2.985,896	7.474,836	15.029,411
6	13,256	97,606	746,747	2.983,383	7.477,897	15.048,826
7	13,260	97,802	747,466	2.986,554	7.478,872	15.048,004
8	13,288	97,404	746,784	2.983,362	7.467,911	15.122,343
9	13,262	97,721	748,878	2.982,535	7.477,316	15.013,400
10	13,334	97,381	747,227	2.982,206	7.482,033	15.063,044
Promedio	13,309	97,556	747,303	2.984,077	7.474,784	15.045,784
Desv. Est.	0,062	0,196	0,813	2,552	4,295	46,331
CV (%)	0,47	0,20	0,11	0,09	0,06	0,31

Cuadro 11: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 4 hilos

OpenMP Base (p=6)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	9,578	65,865	499,890	1.986,506	4.971,813	10.080,769
2	9,541	65,993	500,005	1.987,843	4.973,682	10.082,522
3	9,407	65,618	500,068	1.988,478	4.977,574	10.088,834
4	9,472	65,805	499,893	1.986,293	4.981,597	10.079,555
5	9,483	65,786	499,860	1.985,296	4.977,936	10.093,435
6	9,482	65,722	498,889	1.987,894	4.979,453	10.075,225
7	9,518	66,251	500,035	1.988,407	4.982,998	10.094,598
8	9,434	65,757	499,784	1.988,884	4.981,396	10.089,871
9	9,567	65,771	499,864	1.985,614	4.988,442	10.086,654
10	9,529	65,816	498,267	1.989,138	4.979,043	10.087,054
Promedio	9,501	65,838	499,656	1.987,435	4.979,393	10.085,852
Desv. Est.	0,053	0,165	0,563	1,322	4,465	5,921
CV (%)	0,56	0,25	0,11	0,07	0,09	0,06

Cuadro 12: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 6 hilos

OpenMP Base (p=8)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	7,542	50,480	379,256	1.502,066	3.751,809	7.753,321
2	7,669	50,222	380,369	1.500,359	3.748,881	7.755,870
3	7,721	50,677	381,172	1.502,631	3.748,071	7.751,375
4	7,644	50,715	379,728	1.501,597	3.748,679	7.752,873
5	7,678	50,552	380,985	1.502,592	3.749,868	7.750,247
6	7,634	50,193	381,326	1.502,890	3.751,781	7.751,764
7	7,650	50,362	381,083	1.501,566	3.749,195	7.756,762
8	7,641	50,332	384,013	1.503,845	3.751,672	7.754,768
9	7,547	50,179	379,584	1.503,484	3.747,692	7.754,361
10	7,653	50,563	380,442	1.505,365	3.751,973	7.753,537
Promedio	7,638	50,427	380,796	1.502,639	3.749,962	7.753,488
Desv. Est.	0,052	0,188	1,272	1,320	1,607	1,927
CV (%)	0,69	0,37	0,33	0,09	0,04	0,02

Cuadro 13: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 8 hilos

OpenMP Base (p=12)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	5,907	34,598	258,918	1.014,909	2.558,261	5.478,979
2	5,899	34,476	258,780	1.019,306	2.529,116	5.439,351
3	5,913	34,631	258,892	1.017,249	2.523,906	5.453,944
4	5,860	34,582	258,644	1.013,924	2.521,803	5.440,688
5	5,894	34,689	259,046	1.016,970	2.525,542	5.444,273
6	5,907	34,694	258,573	1.019,746	2.533,260	5.434,957
7	5,867	34,577	258,599	1.018,569	2.527,964	5.485,829
8	5,862	34,628	258,783	1.015,158	2.525,167	5.452,773
9	5,872	34,630	258,652	1.014,502	2.545,479	5.467,133
10	5,891	34,663	258,794	1.014,908	2.547,768	5.447,743
Promedio	5,887	34,617	258,768	1.016,524	2.533,827	5.454,567
Desv. Est.	0,019	0,061	0,145	2,024	11,708	16,420
CV (%)	0,33	0,18	0,06	0,20	0,46	0,30

Cuadro 14: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 12 hilos

Tabla 2 — Promedio y Speedup (ref: Serial + Compiler & Memory Opt)													
Implementación	N=8K Avg (ms)	N=8K Speedup	N=64K Avg (ms)	N=64K Speedup	N=500K Avg (ms)	N=500K Speedup	N=2M Avg (ms)	N=2M Speedup	N=5M Avg (ms)	N=5M Speedup	N=10M Avg (ms)	N=10M Speedup	SpUp Prom.
Serial + Compiler & Memory Opt	0,374	1,0000	2,754	1,0000	22,515	1,0000	90,073	1,0000	227,115	1,0000	453,116	1,0000	1,00
OpenMP Base (p=2)	25,000	0,0150	192,931	0,0143	1.491,735	0,0151	5.975,561	0,0151	14.946,491	0,0152	29.894,829	0,0152	0,01
OpenMP Base (p=4)	13,309	0,0281	97,556	0,0282	747,303	0,0301	2.984,077	0,0302	7.474,784	0,0304	15.045,784	0,0301	0,03
OpenMP Base (p=6)	9,501	0,0394	65,838	0,0418	499,656	0,0451	1.987,435	0,0453	4.979,393	0,0456	10.085,852	0,0449	0,04
OpenMP Base (p=8)	7,638	0,0490	50,427	0,0546	380,796	0,0591	1.502,639	0,0599	3.749,962	0,0606	7.753,488	0,0584	0,06
OpenMP Base (p=12)	5,887	0,0635	34,617	0,0796	258,768	0,0870	1.016,524	0,0886	2.533,827	0,0896	5.454,567	0,0831	0,08

Cuadro 15: Speedup del algoritmo paralelo con OpenMP

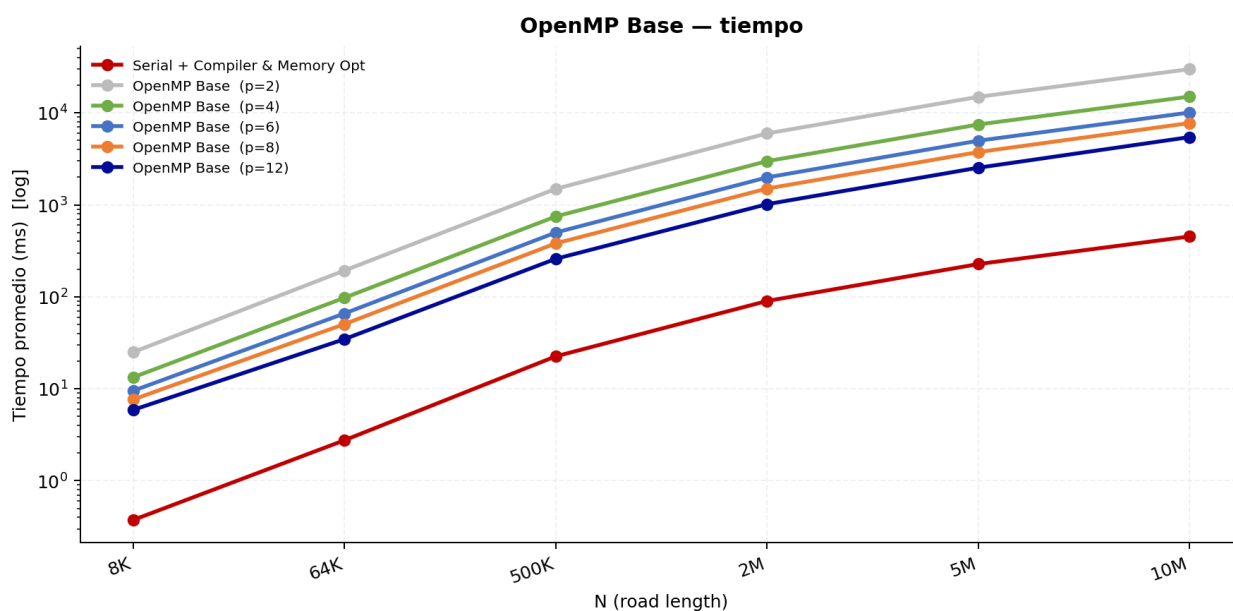


Figura 11: Tiempos de ejecución del algoritmo paralelo con OpenMP

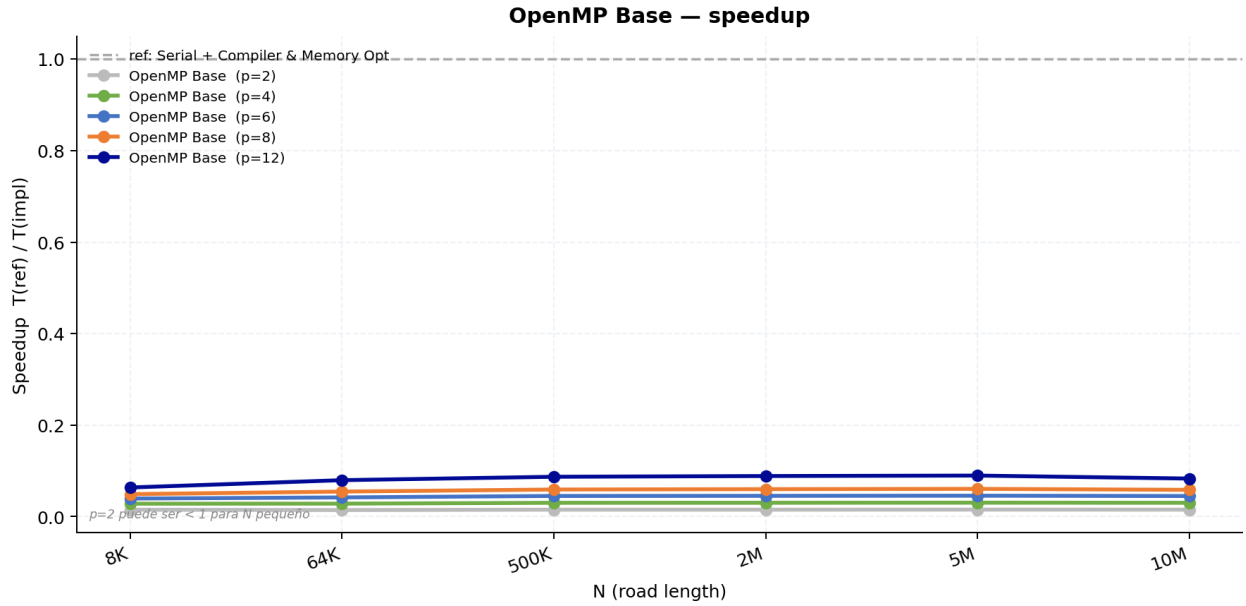


Figura 12: Speedup del algoritmo paralelo con OpenMP

5.3. Resultados de la implementación con OpenMP con optimización por compilador

OpenMP + Compiler Opt (p=2)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	22,003	170,312	1.327,516	5.312,774	13.424,012	26.940,709
2	21,991	170,978	1.327,476	5.311,072	13.418,910	26.942,614
3	22,089	170,671	1.327,267	5.310,482	13.417,516	26.945,627
4	21,976	170,579	1.328,066	5.309,769	13.422,894	26.903,711
5	22,005	170,113	1.327,812	5.309,299	13.424,810	26.937,059
6	21,983	170,235	1.327,161	5.308,831	13.422,443	26.953,658
7	22,000	169,785	1.325,226	5.309,529	13.424,872	26.947,058
8	22,005	169,729	1.328,394	5.315,188	13.434,229	27.003,117
9	21,988	169,798	1.329,579	5.309,572	13.424,488	26.934,693
10	21,999	169,886	1.327,218	5.307,013	13.419,782	26.947,825
Promedio	22,004	170,209	1.327,572	5.310,353	13.423,396	26.945,607
Desv. Est.	0,030	0,406	1,044	2,144	4,382	23,151
CV (%)	0,14	0,24	0,08	0,04	0,03	0,09

Cuadro 16: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 2 hilos y optimización por compilador

OpenMP + Compiler Opt (p=4)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	11,820	85,713	665,816	2.658,023	6.785,234	14.006,056
2	11,892	85,418	667,011	2.656,890	6.790,246	14.032,978
3	11,830	85,973	664,294	2.656,102	6.788,591	14.014,813
4	11,860	86,003	665,120	2.656,633	6.792,056	14.015,559
5	11,905	85,983	664,988	2.657,946	6.791,284	14.051,075
6	11,870	85,538	666,267	2.656,280	6.787,600	14.103,365
7	11,824	85,600	665,738	2.657,483	6.786,641	14.055,442
8	11,806	86,104	665,349	2.656,272	6.808,977	14.036,756
9	11,759	85,704	664,228	2.656,781	6.787,191	14.035,936
10	11,804	85,799	665,597	2.656,734	6.787,010	14.031,212
Promedio	11,837	85,784	665,441	2.656,914	6.790,483	14.038,319
Desv. Est.	0,042	0,216	0,806	0,650	6,498	26,240
CV (%)	0,36	0,25	0,12	0,02	0,10	0,19

Cuadro 17: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 4 hilos y optimización por compilador

OpenMP + Compiler Opt (p=6)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	8,566	57,878	445,065	1.771,755	4.574,193	9.714,072
2	8,545	57,697	444,894	1.774,257	4.576,978	9.719,114
3	8,587	57,568	444,661	1.771,776	4.576,100	9.720,609
4	8,550	57,971	444,847	1.771,275	4.578,214	9.719,405
5	8,494	57,965	444,455	1.772,256	4.576,733	9.709,426
6	8,457	57,616	444,828	1.773,063	4.573,514	9.712,930
7	8,560	57,921	444,637	1.772,445	4.595,515	9.742,273
8	8,623	57,969	445,536	1.772,680	4.572,567	9.711,596
9	8,536	57,833	444,427	1.773,614	4.575,802	9.711,124
10	8,507	57,706	445,195	1.772,461	4.573,796	9.717,643
Promedio	8,543	57,812	444,855	1.772,558	4.577,341	9.717,819
Desv. Est.	0,045	0,146	0,325	0,854	6,288	8,955
CV (%)	0,53	0,25	0,07	0,05	0,14	0,09

Cuadro 18: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 6 hilos y optimización por compilador

OpenMP + Compiler Opt (p=8)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	6,923	43,854	333,368	1.333,810	3.461,871	7.540,479
2	6,940	43,904	333,578	1.331,082	3.465,295	7.543,838
3	7,068	44,163	333,385	1.333,813	3.468,468	7.539,042
4	7,071	44,218	333,401	1.330,505	3.462,537	7.540,197
5	6,976	43,924	333,740	1.330,148	3.467,627	7.545,738
6	6,901	43,772	333,401	1.330,282	3.466,601	7.546,100
7	6,929	43,742	333,714	1.333,808	3.485,076	7.563,139
8	6,918	43,972	333,318	1.329,654	3.465,867	7.542,982
9	6,931	43,777	333,251	1.331,721	3.463,758	7.542,753
10	6,928	43,743	333,262	1.330,078	3.466,717	7.552,519
Promedio	6,958	43,907	333,442	1.331,490	3.467,382	7.545,679
Desv. Est.	0,058	0,161	0,167	1,611	6,237	6,872
CV (%)	0,84	0,37	0,05	0,12	0,18	0,09

Cuadro 19: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 8 hilos y optimización por compilador

OpenMP + Compiler Opt (p=12)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	5,413	30,309	222,337	891,801	2.357,628	5.419,628
2	5,467	30,259	222,339	893,768	2.362,333	5.442,012
3	5,460	30,415	223,595	891,745	2.364,150	5.436,547
4	5,437	30,247	222,606	889,173	2.352,852	5.419,487
5	5,433	30,060	222,828	894,380	2.379,670	5.456,076
6	5,411	30,064	222,390	888,813	2.355,669	5.444,143
7	5,420	30,106	222,837	891,558	2.354,545	5.426,555
8	5,419	30,034	222,767	894,696	2.360,856	5.423,471
9	5,419	30,005	222,464	892,422	2.384,779	5.423,031
10	5,419	30,035	222,572	892,220	2.364,023	5.500,690
Promedio	5,430	30,153	222,674	892,058	2.363,651	5.439,164
Desv. Est.	0,019	0,135	0,357	1,863	10,062	23,549
CV (%)	0,34	0,45	0,16	0,21	0,43	0,43

Cuadro 20: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 12 hilos y optimización por compilador

Tabla 2 — Promedio y Speedup (ref: Serial + Compiler & Memory Opt)													
Implementación	N=8K Avg (ms)	N=8K Speedup	N=64K Avg (ms)	N=64K Speedup	N=500K Avg (ms)	N=500K Speedup	N=2M Avg (ms)	N=2M Speedup	N=5M Avg (ms)	N=5M Speedup	N=10M Avg (ms)	N=10M Speedup	SpUp Prom.
Serial + Compiler & Memory Opt	0,374	1,0000	2,754	1,0000	22,515	1,0000	90,073	1,0000	227,115	1,0000	453,116	1,0000	1,00
OpenMP + Compiler Opt (p=2)	22,004	0,0170	170,209	0,0162	1.327,572	0,0170	5.310,353	0,0170	13.423,396	0,0169	26.945,607	0,0168	0,02
OpenMP + Compiler Opt (p=4)	11,837	0,0316	85,784	0,0321	665,441	0,0338	2.656,914	0,0339	6.790,483	0,0334	14.038,319	0,0323	0,03
OpenMP + Compiler Opt (p=6)	8,543	0,0438	57,812	0,0476	444,855	0,0506	1.772,558	0,0508	4.577,341	0,0496	9.717,819	0,0466	0,05
OpenMP + Compiler Opt (p=8)	6,958	0,0537	43,907	0,0627	333,442	0,0675	1.331,490	0,0676	3.467,382	0,0655	7.545,679	0,0600	0,06
OpenMP + Compiler Opt (p=12)	5,430	0,0689	30,153	0,0913	222,674	0,1011	892,058	0,1010	2.363,651	0,0961	5.439,164	0,0833	0,09

Cuadro 21: Speedup del algoritmo paralelo con OpenMP y optimización por compilador

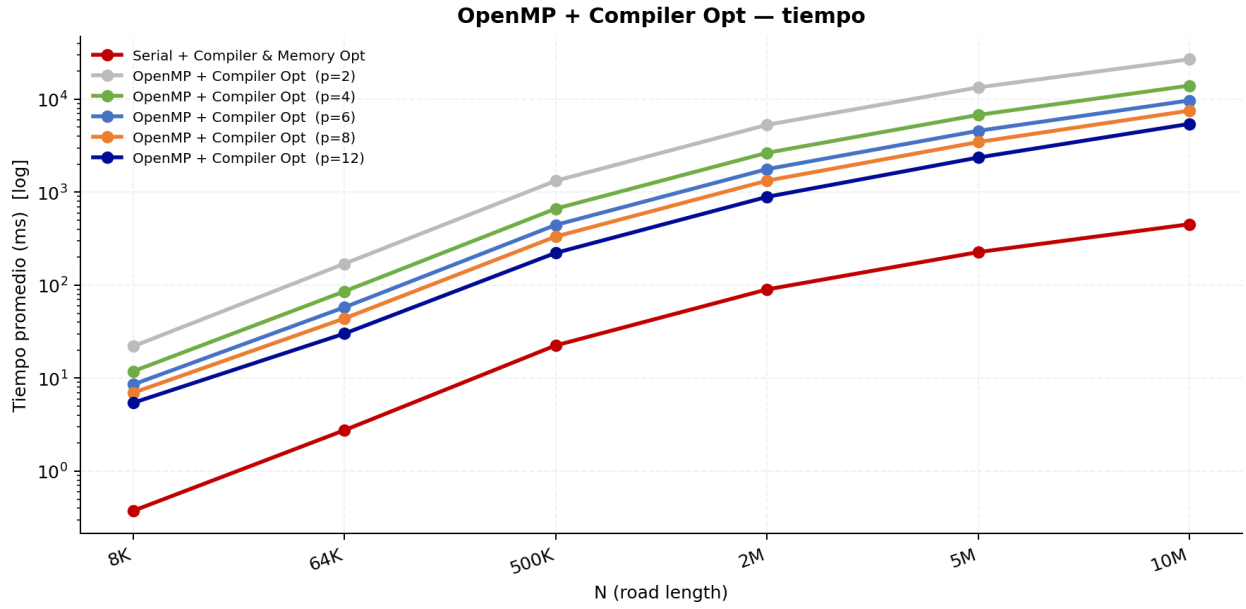


Figura 13: Tiempos de ejecución del algoritmo paralelo con OpenMP y optimización por compilador

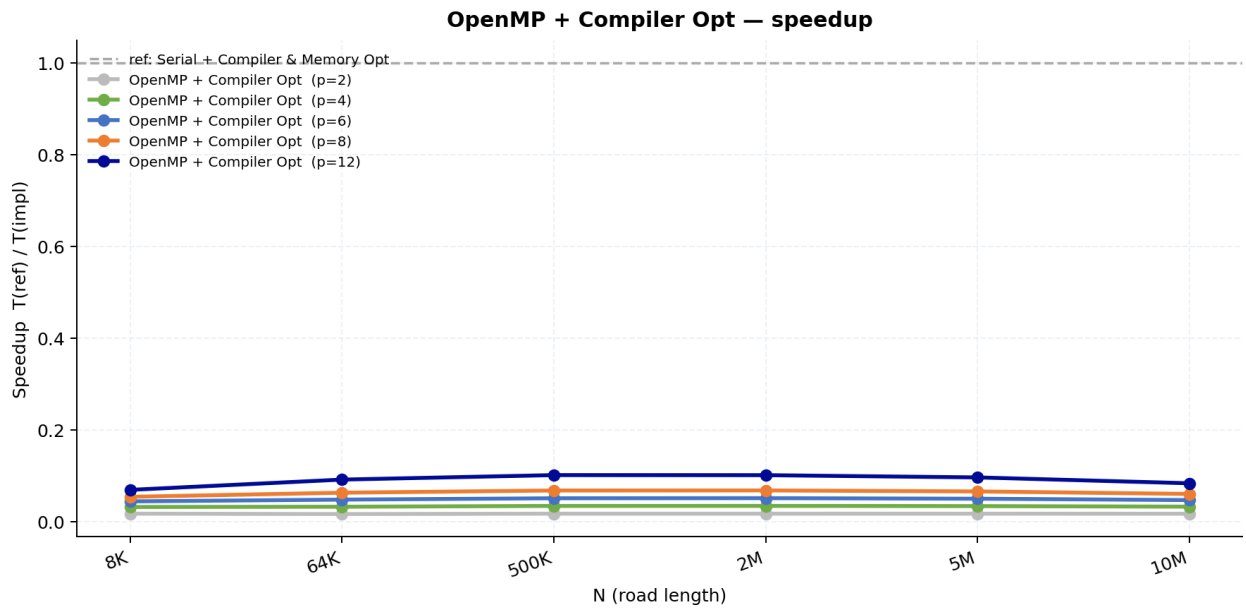


Figura 14: Speedup del algoritmo paralelo con OpenMP y optimización por compilador

5.4. Resultados de la implementación con OpenMP con optimización de memoria y compilador

OpenMP + Compiler & Memory Opt (p=2)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	0,932	3,121	21,541	84,045	207,153	413,590
2	0,984	3,092	21,453	83,342	207,008	413,192
3	0,963	3,129	21,690	83,283	206,385	412,677
4	0,985	3,101	21,468	83,303	207,038	412,947
5	0,993	3,133	21,555	83,326	207,039	411,989
6	0,954	3,109	21,510	83,299	206,542	412,493
7	0,960	3,130	21,594	83,555	206,606	412,702
8	0,985	3,113	21,770	83,238	207,286	414,243
9	0,980	3,114	21,611	83,284	206,513	412,787
10	0,971	3,140	21,467	83,279	207,247	412,058
Promedio	0,971	3,118	21,566	83,395	206,882	412,868
Desv. Est.	0,018	0,014	0,098	0,232	0,318	0,645
CV (%)	1,82	0,46	0,45	0,28	0,15	0,16

Cuadro 22: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 2 hilos y optimización de memoria y compilador

OpenMP + Compiler & Memory Opt (p=4)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	0,971	1,993	11,018	42,230	105,034	206,916
2	0,865	1,973	11,199	42,110	104,915	206,841
3	0,968	1,981	11,122	42,231	104,916	206,854
4	0,826	1,937	11,024	42,066	105,216	206,437
5	0,825	1,927	11,156	42,197	104,992	206,925
6	0,959	1,968	10,969	42,228	105,046	206,935
7	0,966	1,988	11,018	42,278	104,666	206,570
8	0,909	1,972	11,101	42,486	105,016	207,371
9	0,956	1,895	11,046	42,062	104,633	206,907
10	0,959	1,985	11,187	42,199	104,758	206,582
Promedio	0,920	1,962	11,084	42,209	104,919	206,834
Desv. Est.	0,057	0,030	0,076	0,116	0,174	0,247
CV (%)	6,18	1,54	0,69	0,28	0,17	0,12

Cuadro 23: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 4 hilos y optimización de memoria y compilador

OpenMP + Compiler & Memory Opt (p=6)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	1,000	1,626	7,767	28,914	70,024	138,961
2	0,986	1,621	7,748	28,887	69,842	138,242
3	0,989	1,619	7,742	28,768	69,807	138,261
4	0,948	1,610	7,745	28,884	70,047	139,573
5	0,980	1,633	7,783	28,649	69,803	137,863
6	0,974	1,621	7,744	28,930	69,834	138,269
7	0,978	1,627	7,887	28,986	69,987	137,983
8	0,976	1,668	7,771	28,841	69,839	138,543
9	0,986	1,621	7,773	28,900	69,715	137,973
10	0,988	1,627	7,813	28,953	69,729	138,316
Promedio	0,980	1,627	7,777	28,871	69,863	138,398
Desv. Est.	0,013	0,015	0,042	0,093	0,111	0,492
CV (%)	1,33	0,91	0,54	0,32	0,16	0,36

Cuadro 24: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 6 hilos y optimización de memoria y compilador

OpenMP + Compiler & Memory Opt (p=8)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	1,076	1,501	6,076	21,976	53,035	105,503
2	1,032	1,484	6,086	21,835	53,146	105,562
3	1,022	1,485	6,097	22,021	53,026	105,508
4	1,020	1,480	6,076	22,094	52,953	105,347
5	1,037	1,467	6,025	22,007	52,894	104,974
6	1,033	1,494	6,059	21,932	53,162	104,975
7	1,069	1,482	6,104	22,058	53,431	105,079
8	1,036	1,491	6,037	21,844	53,023	104,964
9	1,025	1,465	6,064	22,027	53,073	105,071
10	1,022	1,480	6,070	21,936	53,053	105,136
Promedio	1,037	1,483	6,069	21,973	53,080	105,212
Desv. Est.	0,019	0,011	0,023	0,082	0,139	0,230
CV (%)	1,80	0,71	0,39	0,37	0,26	0,22

Cuadro 25: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 8 hilos y optimización de memoria y compilador

OpenMP + Compiler & Memory Opt (p=12)						
Rep	N=8K	N=64K	N=500K	N=2M	N=5M	N=10M
1	1,185	1,453	4,457	15,085	36,308	70,618
2	1,137	1,458	4,517	15,067	36,252	70,722
3	1,172	1,429	4,453	15,063	36,328	70,736
4	1,119	1,397	4,471	15,240	36,342	70,676
5	1,132	1,377	4,457	15,021	36,130	70,426
6	1,124	1,390	4,521	14,968	35,981	70,457
7	1,125	1,388	4,466	15,011	36,420	70,554
8	1,131	1,411	4,472	14,997	36,056	70,456
9	1,129	1,419	4,456	14,936	35,907	70,748
10	1,119	1,413	4,450	14,990	36,056	70,484
Promedio	1,137	1,413	4,472	15,038	36,178	70,588
Desv. Est.	0,021	0,026	0,025	0,080	0,166	0,121
CV (%)	1,89	1,82	0,55	0,54	0,46	0,17

Cuadro 26: Tiempos de ejecución del algoritmo paralelo con OpenMP utilizando 12 hilos y optimización de memoria y compilador

Tabla 2 — Promedio y Speedup (ref: Serial + Compiler & Memory Opt)													
Implementación	N=8K Avg (ms)	N=8K Speedup	N=64K Avg (ms)	N=64K Speedup	N=500K Avg (ms)	N=500K Speedup	N=2M Avg (ms)	N=2M Speedup	N=5M Avg (ms)	N=5M Speedup	N=10M Avg (ms)	N=10M Speedup	SpUp Prom.
Serial + Compiler & Memory Opt	0,374	1,0000	2,754	1,0000	22,515	1,0000	90,073	1,0000	227,115	1,0000	453,116	1,0000	1,00
OpenMP + Compiler & Memory Opt (p=2)	0,971	0,3853	3,118	0,8832	21,566	1,0440	83,395	1,0801	206,882	1,0978	412,868	1,0975	0,93
OpenMP + Compiler & Memory Opt (p=4)	0,920	0,4063	1,962	1,4037	11,084	2,0313	42,209	2,1340	104,919	2,1647	206,834	2,1907	1,72
OpenMP + Compiler & Memory Opt (p=6)	0,980	0,3814	1,627	1,6923	7,777	2,8950	28,871	3,1198	69,863	3,2509	138,398	3,2740	2,44
OpenMP + Compiler & Memory Opt (p=8)	1,037	0,3606	1,483	1,8571	6,069	3,7096	21,973	4,0993	53,080	4,2788	105,212	4,3067	3,10
OpenMP + Compiler & Memory Opt (p=12)	1,137	0,3288	1,413	1,9483	4,472	5,0347	15,038	5,9898	36,178	6,2777	70,588	6,4192	4,33

Cuadro 27: Speedup del algoritmo paralelo con OpenMP y optimización de memoria y compilador

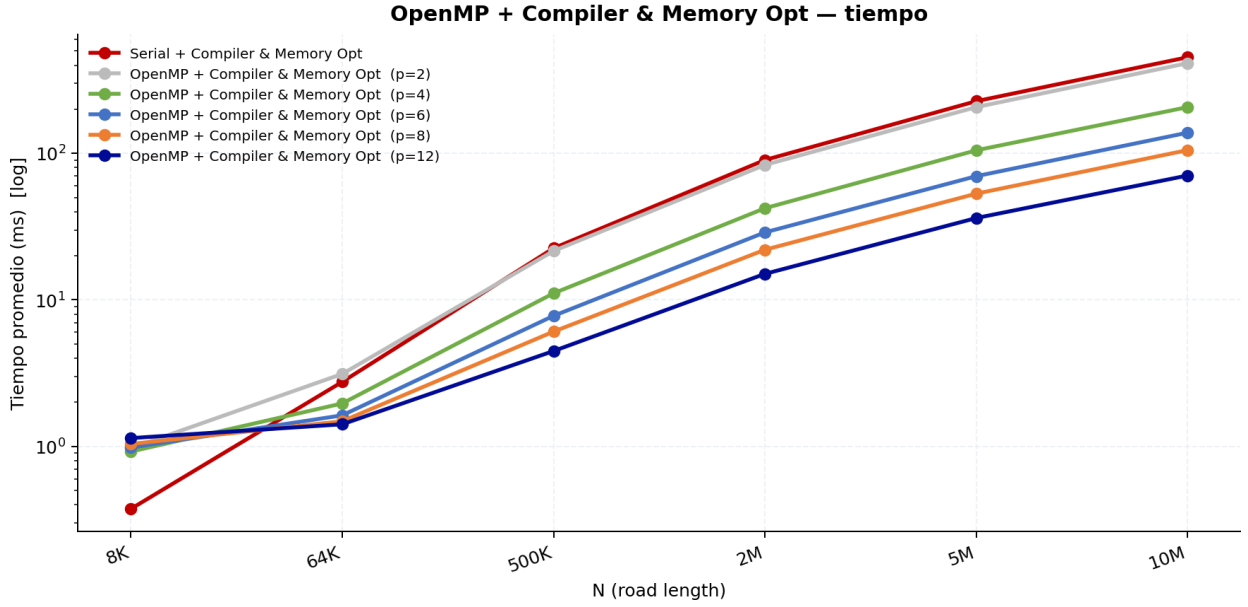


Figura 15: Tiempos de ejecución del algoritmo paralelo con OpenMP y optimización de memoria y compilador

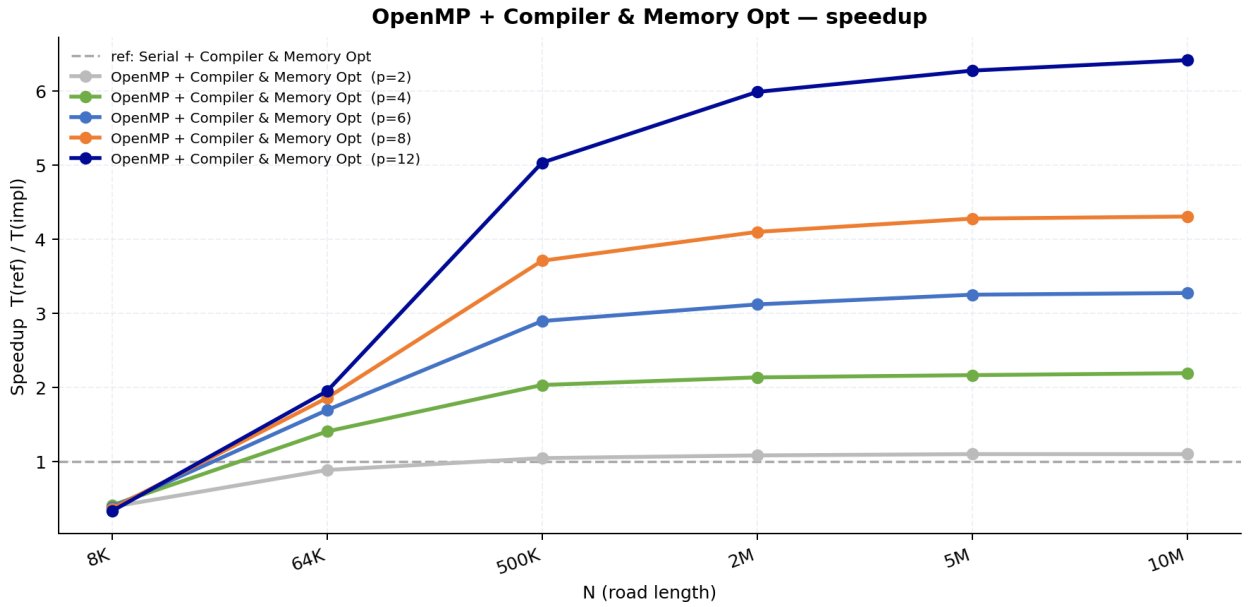


Figura 16: Speedup del algoritmo paralelo con OpenMP y optimización de memoria y compilador

6. Conclusiones

1. El resultado más importante del experimento es que la implementación secuencial optimizada tanto en memoria como por el compilador alcanza un speedup cercano a 65x

frente a la implementación secuencial base, y lo hace de manera muy estable para todos los tamaños del problema. Para un caso representativo de $N = 10$ millones, el tiempo de ejecución se reduce de 29,608 ms a solo 453 ms usando un único hilo, lo cual evidencia que una buena optimización puede ser más efectiva que aumentar el paralelismo.

Lo más interesante de este resultado es que ninguna de las dos optimizaciones funciona bien por sí sola. Cuando solo se aplica la optimización del compilador, la mejora es pequeña, alrededor de 1.09 veces. Cuando solo se aplica la optimización de memoria, el rendimiento incluso empeora ligeramente frente a la versión base. Esto ocurre porque cada optimización ataca un problema distinto, pero ninguna es suficiente por sí sola.

En el caso de la optimización del compilador aplicada sobre la versión base, cada celda del autómata se almacena como un entero de 32 bits (`int`). Aunque el compilador puede acelerar parcialmente el bucle, un registro AVX2 de 256 bits solo puede contener 8 enteros, por lo que cada instrucción SIMD procesa apenas 8 celdas en paralelo. El potencial de vectorización queda así muy limitado por la anchura del tipo de dato.

Por otro lado, la optimización de memoria reduce el tamaño de cada celda de 4 bytes a 1 byte (`uint8_t`), aprovechando que cada celda solo puede valer 0 o 1, y además fusiona los dos pases del algoritmo en uno solo. Sin embargo, si el compilador no optimiza el código, la lógica adicional necesaria para este pase único resulta más costosa que el enfoque base con dos buffers simples, lo que explica por qué esta versión no mejora el rendimiento por sí sola.

La gran ganancia aparece cuando ambas optimizaciones se combinan. Con celdas de tipo `uint8_t`, un registro AVX2 de 256 bits contiene 32 celdas en lugar de 8, lo que cuadruplica el número de celdas procesadas por instrucción SIMD. Sumado al factor de 2x que aporta la fusión en un único pase, el efecto combinado sobre el rendimiento computacional alcanza aproximadamente 8x solo en densidad de instrucciones. Esto es consistente con el perfilado de accesos a memoria realizado con `perf mem`, que mostró una tasa de aciertos en L1 superior al 56 % incluso en la versión base, indicando que la jerarquía de caché no era el cuello de botella principal. La ganancia observada proviene fundamentalmente de procesar más trabajo útil por instrucción, no de reducir fallos de caché.

2. Al comparar la implementación secuencial optimizada por memoria y compilador (453 ms, un hilo) contra la implementación paralela con OpenMP sin optimización de memoria usando 12 hilos (5 454 ms), se observa que la versión secuencial optimizada es aproximadamente 12 veces más rápida que la versión paralela de máxima concurrencia sin optimización de memoria. Este resultado reproduce el mismo patrón observado en

el proyecto de Jacobi: cuando el cuello de botella principal es el acceso a memoria, agregar hilos sin mejorar la localidad de datos y el uso de caché únicamente incrementa la presión sobre el bus de memoria.

La implementación paralela con OpenMP sin optimización de memoria usando 2 hilos para $N = 10$ millones arroja un tiempo de 29 894 ms, prácticamente idéntico al de la versión secuencial base (29 608 ms), lo cual indica que con pocos hilos el overhead de sincronización anula completamente cualquier posible ganancia. Solo a partir de 6 hilos el speedup comienza a ser significativo, cercano a 3 veces, alcanzando aproximadamente 5.4x con 12 hilos, lo que corresponde a una eficiencia paralela del 45 %.

3. Cuando se combinan todas las optimizaciones, es decir, paralelización con OpenMP junto con optimización de memoria y compilador, utilizando 12 hilos, el speedup total respecto a la implementación secuencial base alcanza aproximadamente 420x para $N = 10$ millones y si la comparamos con la versión secuencial optimizada, el speedup es de 4.33x. El escalado paralelo de esta variante es saludable: cada duplicación del número de hilos reduce aproximadamente a la mitad el tiempo de ejecución, y la eficiencia paralela se mantiene estable alrededor del 53 % para tamaños de problema grandes, lo cual es un resultado muy bueno para una carga de trabajo dominada por accesos secuenciales a memoria.

Para tamaños pequeños, como $N = 8$ mil, el comportamiento colapsa, todas las configuraciones de hilos presentan tiempos cercanos a 1 ms, independientemente del paralelismo, debido a que el problema cabe completamente en la caché L2 o L3 y el overhead de OpenMP domina el tiempo total.

4. La implementación paralela con OpenMP optimizada únicamente por compilador aporta muy poco beneficio adicional frente a la implementación paralela con OpenMP sin optimizaciones. Para $N = 10$ millones con 12 hilos, la versión base tarda 5 454 ms mientras que la versión optimizada por compilador tarda 5 439 ms, una diferencia inferior al 0.3 %. El compilador no puede vectorizar de manera efectiva un bucle paralelo de OpenMP que opera sobre la estructura de datos original, y el cuello de botella continúa siendo el acceso a memoria. Este resultado refuerza que la ganancia real de la optimización de compilador está fuertemente condicionada al layout de memoria.
5. Las implementaciones paralelas con OpenMP sin optimización y con OpenMP optimizada únicamente por compilador no logran superar en ningún caso a la mejor implementación secuencial. Incluso con $p = 12$ hilos, el speedup máximo obtenido es de apenas 0.10 veces, lo que implica que estas variantes son entre 10 y 12 veces más lentas

que la implementación secuencial optimizada por memoria y compilador. Este comportamiento se explica porque el paralelismo introduce overhead adicional asociado a la creación y sincronización de hilos, así como mayor tráfico de caché, pero no resuelve el problema principal, los arreglos siguen almacenados como enteros y el conjunto de datos continúa saturando el acceso a memoria.

6. La implementación paralela con OpenMP optimizada tanto por memoria como por compilador es la única variante paralela que logra superar a la referencia secuencial, y esto solo ocurre a partir de tamaños de problema de $N = 64$ mil con $p \geq 4$ hilos. Para $N = 8$ mil, el speedup cae por debajo de 1 incluso utilizando 12 hilos, lo que indica claramente que el overhead asociado a OpenMP es mayor que el beneficio del paralelismo cuando el conjunto de datos es pequeño.
7. El speedup máximo alcanzado por la implementación paralela con OpenMP optimizada por memoria y compilador es de aproximadamente 6.4 veces usando 12 hilos para $N = 10$ millones. Aunque esta mejora es real, resulta modesta si se considera el número de hilos disponibles. La eficiencia paralela se mantiene entre el 53 % y el 55 % para tamaños de problema mayores o iguales a $N = 500$ mil, lo que indica que el algoritmo no escala de forma lineal con el número de hilos. Este comportamiento es típico de cargas de trabajo limitadas por el acceso a memoria, donde el ancho de banda de la memoria principal es el recurso compartido y dominante.
8. Para $N = 8$ mil y $p = 12$ hilos, la eficiencia paralela es de apenas 2.7 %, lo que confirma que el problema es demasiado pequeño para amortizar el costo del paralelismo. A medida que el tamaño del problema aumenta, la eficiencia se estabiliza alrededor del 54 % independientemente del número de hilos utilizados. Esto sugiere que el sistema alcanza rápidamente el límite del ancho de banda de memoria compartida entre los núcleos y que no es posible escalar más allá sin modificar la forma en que se accede y se organiza la información en memoria.
9. La velocidad media del flujo de vehículos es prácticamente idéntica en todas las implementaciones, con un valor cercano a 0.989 para tamaños grandes y densidad $\rho = 0,5$. Esto confirma que todas las variantes producen el mismo resultado físicamente correcto y que las optimizaciones aplicadas no alteran el resultado del autómata celular, sino únicamente su rendimiento computacional.

Referencias

- Ben-Ari, M. (2006). *Principles of Concurrent and Distributed Programming* (2nd). Addison-Wesley.
- Chapman, B., Jost, G., & van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.
- Dagum, L., & Menon, R. (1998). OpenMP: An Industry-Standard API for Shared-Memory Programming. *IEEE Computational Science and Engineering*, 5(1), 46-55. <https://doi.org/10.1109/99.660313>
- Drepper, U. (2007). *What Every Programmer Should Know About Memory* (Technical Report). Red Hat, Inc. <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>
- Gustafson, J. L. (1988). Reevaluating Amdahl's Law. *Communications of the ACM*, 31(5), 532-533. <https://doi.org/10.1145/42411.42415>
- Hager, G., & Wellein, G. (2010). *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
- Nagel, K., & Schreckenberg, M. (1992). A cellular automaton model for freeway traffic. *Journal de Physique I*, 2(12), 2221-2229. <https://doi.org/10.1051/jp1:1992277>
- Patterson, D. A., & Hennessy, J. L. (2017). *Computer Architecture: A Quantitative Approach* (6th). Morgan Kaufmann.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th). Wiley.
- Williams, S., Waterman, A., & Patterson, D. A. (2009). Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65-76. <https://doi.org/10.1145/1498765.1498785>
- Wolfram, S. (1984). Universality and complexity in cellular automata. *Physica D: Nonlinear Phenomena*, 10(1-2), 1-35. [https://doi.org/10.1016/0167-2789\(84\)90245-8](https://doi.org/10.1016/0167-2789(84)90245-8)
- Yi, L., Li, C., & Guo, J. (2020). CPI for Runtime Performance Measurement: The Good, the Bad, and the Ugly. *IEEE International Symposium on Workload Characterization (IISWC 2020)*, 15-25. <https://doi.org/10.1109/IISWC50251.2020.00011>